

UNIVERSIDAD EL BOSQUE
FACULTAD DE INGENIERÍA
PROYECTO FINAL



ASIGNATURA:

Ingeniería de Software II

DOCENTE:

Wilson Rojas Reales

ELABORADO POR:

Samuel Julián Rodríguez Chávez

Natalia Vanesa Martínez Rodríguez

Abraham Pesca Prada

ÍNDICE

1. Documento de arquitectura.....	3
a. Objetivo del sistema a construir:.....	3
b. Justificación del uso de microservicios.....	3
c. Exposición del microservicio (API REST, API Gateway, Nube).....	4
d. Atributos de calidad abordados.....	5
d. Diagrama de componentes y comunicación entre servicios.....	9
f. Diagrama de despliegue.....	10
2. Diseño de los Microservicios. Por cada microservicio:.....	10
a. Nombre y propósito.....	10
b. Interfaces expuestas (endpoints REST).....	10
c. Modelo de datos (sin relaciones, sin índices, con relaciones, con índices).....	14
d. Diccionario de datos.....	20
e. Diagrama de clases (si aplica).....	30
f. Dependencias externas (bases de datos, otros servicios).....	31
3. Código fuente funcional.....	33
a. El código fuente deberá entregarse mediante una USB o repositorio en la nube... 33	
b. Código fuente de cada microservicio.....	33
c. API REST documentada con Swagger/Open API.....	33
d. Informe de pruebas unitarias y de integración (si aplica). Puede utilizar herramientas como: Junit, Postman, etc.....	35
4. API Gateway.....	42
a. Configuración de enrutamiento hacia los microservicios.....	42
b. Seguridad básica (token, autenticación).....	42
c. Documentación de endpoints expuestos por el gateway (NO APLICA).....	43
5. Despliegue y Acceso (en este ítem, el grupo deberá demostrar el funcionamiento del proyecto).....	43
a. Microservicios desplegados (localmente, Render, Heroku, Máquinas virtuales, etc).....	43
b. Evidencia del funcionamiento (evidencias gráficas que deberá colocar en el informe a entregar).....	45
6. Documentación:.....	58
a. Manual de Usuario (si existe interfaz gráfica).....	58
b. Manual de instalación y despliegue.....	59
7. Estimación del producto.....	59

1. Documento de arquitectura

a. Objetivo del sistema a construir:

El objetivo principal del sistema web multiplataforma es automatizar el proceso de gestión de cotizaciones y pedidos dentro de la cadena local de droguerías SophyFarm. Este sistema tiene como propósito sistematizar las operaciones que actualmente se realizan de manera manual, lo que permitirá reducir el riesgo de errores y mejorar significativamente la eficiencia operativa.

El sistema está diseñado específicamente para que los vendedores gestionen de manera eficiente las cotizaciones solicitadas por los clientes solo a crédito, teniendo la posibilidad que estas se conviertan en pedidos de forma ágil. Solo se manejan pedidos a partir de una cotización ya creada.

No solo se busca permitir la creación, modificación y seguimiento de cotizaciones y pedidos, sino también proporcionar la información necesaria para la toma de decisiones estratégicas. Esto se logrará a través de reportes que ofrecerán datos clave sobre el desempeño de las cotizaciones y pedidos, así como el comportamiento de los clientes, facilitando así la toma de decisiones informadas que beneficien al negocio.

A medida que avance el desarrollo del sistema, se integrarán de manera fluida las operaciones de SophyFarm, permitiendo la consulta en tiempo real de productos, el control de stock entre las diferentes sucursales y la correcta gestión de los pedidos, sin importar la ubicación o disponibilidad de los productos en cada una de ellas.

El sistema también incluirá una herramienta accesible y fácil de usar para registrar el historial de cotizaciones y pedidos, mejorando la visibilidad del negocio y facilitando el seguimiento detallado de cada transacción de manera clara y ordenada.

En resumen, el sistema tiene como objetivo transformar tecnológicamente los procesos de cotización y pedidos en SophyFarm, los cuales actualmente son manuales en un 80%. Frente al auge tecnológico de los mercados competitivos, se espera que, con el desarrollo de este sistema, se logren evitar errores que produce el registro manual, se reduzca el tiempo de procesamiento de ventas y se proporcione información valiosa para la gestión y el crecimiento del negocio.

b. Justificación del uso de microservicios.

La elección de la arquitectura de microservicios con relación al módulo de cotizaciones y pedidos del sistema web multiplataforma responde a las necesidades específicas del proyecto y los retos que enfrenta SophyFarm en su proceso de automatización y optimización de ventas.

Independencia y Flexibilidad en el Desarrollo

El módulo de cotizaciones y pedidos requiere una gestión eficiente de diferentes procesos que, aunque relacionados, son independientes en su funcionamiento. Usar microservicios permite que cada funcionalidad (ya sea la creación de cotizaciones, el registro de pedidos o la gestión de stock) esté aislada en un servicio independiente, lo que facilita el desarrollo, la implementación de nuevas características y la modificación de procesos sin afectar el funcionamiento global del sistema.

Escalabilidad Adaptada a la Demanda

A medida que SophyFarm crece y las transacciones aumentan, la arquitectura de microservicios permitirá escalar los servicios de manera independiente. Por ejemplo, si se incrementa la cantidad de pedidos, solo se escalará el microservicio de pedidos sin necesidad de escalar todo el sistema.

Resiliencia y Mantenibilidad

El enfoque basado en microservicios mejora la resiliencia del sistema, ya que un fallo en un microservicio no afectará a los demás. Esto resulta fundamental para un sistema de gestión de ventas como el que requiere SophyFarm, el cual debe ser altamente disponible y operativo en todo momento. Además, la mantenibilidad del sistema se ve favorecida, pues las actualizaciones o correcciones en un microservicio específico pueden realizarse sin interrumpir el servicio completo.

Adaptabilidad a Nuevas Necesidades

El sistema debe ser capaz de adaptarse a posibles futuros cambios en el negocio, por ejemplo: la incorporación de nuevos tipos de productos, la expansión de sucursales o la integración con otros sistemas externos. La arquitectura de microservicios permite esta flexibilidad, lo que facilita la evolución del sistema a medida que SophyFarm amplía sus operaciones sin requerir reestructuraciones mayores.

c. Exposición del microservicio (API REST, API Gateway, Nube).

Para el desarrollo del módulo de cotizaciones y pedidos para SophyFarm se optó por la exposición de microservicios a través de APIs REST como la principal forma de interacción entre los distintos componentes del sistema. Esta decisión responde a las necesidades del proyecto, el cual logra mantener un enfoque simplificado, eficiente y fácil de implementar.

Uso de API REST

La arquitectura de microservicios se basa en la exposición de cada servicio a través de endpoints REST, los cuales permiten que cada microservicio funcione de manera independiente y se comunique de forma eficiente con el resto del sistema.

En el caso del módulo a desarrollar (cotizaciones y pedidos) los endpoints REST serán utilizados para realizar todas las operaciones esenciales del sistema, como:

- Crear, consultar, actualizar, y eliminar clientes, ya sean clientes naturales o empresas (razón social).
- Crear, consultar, actualizar y eliminar tanto cotizaciones como pedidos.
- Gestionar los pedidos (incluyendo su estado y creación directamente desde el ID de una cotización existente).
- Enviar una notificación de la cotización o pedido en estado “**Confirmado**” al correo electrónico y a WhatsApp.
- Consultar el stock de los productos.

El análisis de la elección ha dejado en detalle la evidencia de múltiples ventajas:

- **Simplicidad:** Las APIs REST son fáciles de desarrollar, consumir y mantener. Además, permiten una integración directa con la interfaz de usuario y con cualquier otro sistema que pueda necesitar interactuar con el sistema de SophyFarm en el futuro.
- **Flexibilidad:** REST permite que cada microservicio esté aislado, lo que facilita la escalabilidad y la adaptación a nuevas funcionalidades sin afectar el resto del sistema.
- **Compatibilidad:** Dado que REST es ampliamente adoptado por este tipo de sistemas de información, se logra garantizar la compatibilidad con una amplia variedad de tecnologías y plataformas, facilitando la interacción entre los diferentes componentes del sistema.

Despliegue Local y Controlado

Al mantener el sistema en un entorno controlado (en el caso una máquina virtual DEBIAN 11), se logra obtener un control más directo sobre la seguridad, copias de seguridad y acceso a los datos.

d. Atributos de calidad abordados

1. Disponibilidad

Relevancia:

La alta disponibilidad es fundamental en el sistema, ya que los vendedores deben tener acceso continuo para gestionar cotizaciones, pedidos y productos sin interrupciones.

Microservicios involucrados:

CRUD Cotizaciones: Este servicio debe estar siempre disponible para permitir que los vendedores gestionen cotizaciones sin interrupciones.

CRUD Pedidos: Una vez que una cotización es aceptada, los pedidos deben generarse sin demoras. Si este servicio falla, se pierde la oportunidad de transformar cotizaciones en pedidos.

CRUD Clientes: Los clientes deben poder ser consultados y gestionados en tiempo real.

Justificación:

La disponibilidad es un atributo de calidad muy importante para cualquier sistema que tenga como objetivo realizar transacciones comerciales en tiempo real, en este caso para el módulo de cotizaciones y pedidos. Los vendedores deben poder acceder al sistema para registrar, consultar y convertir cotizaciones en pedidos en cualquier momento. Dado que el sistema reemplaza un proceso que previamente se hacía manualmente en Excel, es vital que los vendedores puedan acceder al sistema en todo momento, tanto en presencia física como en llamada. En caso tal, si el sistema no está disponible o falla, se puede llegar a perder la oportunidad de cerrar una venta y puede generar una mala experiencia para el cliente. Además, en un sistema que involucra varias sucursales, la disponibilidad asegura que todas las operaciones, como la validación de stock entre diferentes sedes, puedan ejecutarse de manera continua y sin interrupciones.

2. Rendimiento**Relevancia:**

En un sistema donde los vendedores interactúan constantemente con el sistema (consultando productos, realizando cotizaciones y pedidos), el rendimiento debe ser ágil y sin retrasos que lleguen a generar pérdidas significativas.

Microservicios involucrados:

Productos: La consulta y visualización de productos debe ser rápida, ya que los vendedores necesitan ver los productos disponibles para cotizar en tiempo real.

CRUD Cotizaciones y CRUD Pedidos: La creación y gestión de cotizaciones y pedidos debe ser instantánea, ya que un retraso afectaría la eficiencia del vendedor.

Justificación:

En el módulo de cotizaciones y pedidos, las búsquedas de productos, la validación de disponibilidad de stock y la creación o modificación de cotizaciones deben ser procesos que puedan ser ejecutados rápidamente, ya que cualquier retraso en estos procesos podría afectar la experiencia del usuario y la eficiencia de los vendedores. El vendedor de la droguería debe obtener resultados de cualquier acción que requiera de manera casi instantánea para brindar un servicio ágil a los clientes. Un rendimiento deficiente puede generar tiempos de espera continuos, que pueden conllevar a la frustración tanto de los empleados como de los clientes. Además, como el sistema se usará tanto en la droguería presencial como por llamada, cualquier retraso en las operaciones podría aumentar el tiempo de atención al cliente, especialmente en llamadas donde la eficiencia es crucial.

3. Seguridad

Relevancia:

Dado que el sistema maneja información sensible (detalles personales de los clientes, precios de productos, datos comerciales, etc.), la seguridad debe ser una prioridad para proteger la confidencialidad e integridad de los datos.

Microservicios involucrados:

Autenticación: Se utiliza para verificar los roles de los usuarios (vendedores, administradores) antes de permitirles realizar acciones críticas (como la creación de cotizaciones o pedidos).

CRUD Clientes y CRUD Pedidos: Los vendedores solo pueden gestionar sus propios clientes y pedidos, y los datos deben ser protegidos en todo momento, especialmente en la transmisión de la información.

Notificaciones: Las notificaciones enviadas a través de correo electrónico o WhatsApp también deben estar cifradas para proteger los datos del cliente.

Justificación:

La autenticación basada en JWT (JSON Web Tokens) garantiza que solo los usuarios autorizados puedan interactuar con los servicios, y la comunicación entre microservicios puede ser cifrada utilizando HTTPS. Además, las credenciales y la información personal se gestionan de manera segura mediante la implementación de controles de acceso adecuados en cada microservicio.

4. Usabilidad

Relevancia:

Aunque no es tan crítico como otros atributos, la usabilidad sigue siendo importante para asegurar que los usuarios (vendedores y administradores) puedan interactuar con el sistema de una forma fluida.

Microservicios involucrados:

CRUD Cotizaciones, CRUD Pedidos, CRUD Clientes: Todos estos servicios deben tener interfaces claras y fáciles de usar, ya que el sistema es utilizado por vendedores que deben completar tareas rápidamente.

Justificación:

En este caso, aunque la usabilidad es menos crítica que en un sistema orientado a un uso del cliente, sigue siendo relevante. El sistema debe ser fácil de usar para los vendedores, quienes realizan múltiples tareas de manera simultánea y deben hacerlo sin mayor dificultad. Dado que el sistema reemplaza un proceso manual que antes se realizaba en Excel, los vendedores

deben poder adaptarse rápidamente al sistema para continuar las determinadas tareas que hacían antes manualmente sin perder tiempo. Esto se refleja directamente en la eficiencia operativa, debido a que, entre más facilidad tenga el vendedor al usar el sistema, más rápido podrá atender las solicitudes de los clientes y procesar las transacciones.

5. Mantenibilidad

Relevancia:

La mantenibilidad es fundamental para asegurarse de que el sistema pueda evolucionar, adaptarse a cambios y corregir errores sin afectar otras áreas.

Microservicios involucrados:

Todos los microservicios: Debido a la naturaleza independiente de cada servicio, cualquier cambio o corrección en un microservicio no afecta al resto del sistema. Si es necesario añadir nuevas funcionalidades o modificar la lógica de negocio, esto puede hacerse de manera aislada.

Justificación:

La arquitectura de microservicios facilita la mantenibilidad, ya que cada componente se puede actualizar de forma independiente sin necesidad de detener o afectar otros servicios. Además, los contenedores y herramientas como Docker permiten que el sistema sea más fácil de gestionar y actualizar, sin generar tiempo de inactividad.

6. Escalabilidad

Relevancia:

La escalabilidad es crucial para soportar el crecimiento del negocio, como un mayor número de clientes, cotizaciones y pedidos. A medida que la empresa crezca, el sistema debe ser capaz de manejar cargas más grandes.

Microservicios involucrados:

CRUD Cotizaciones, CRUD Pedidos: Ambos servicios pueden escalarse de manera independiente para manejar mayores volúmenes de transacciones.

CRUD Clientes: A medida que la base de clientes crezca, este servicio debe ser capaz de gestionar la información sin afectar el rendimiento.

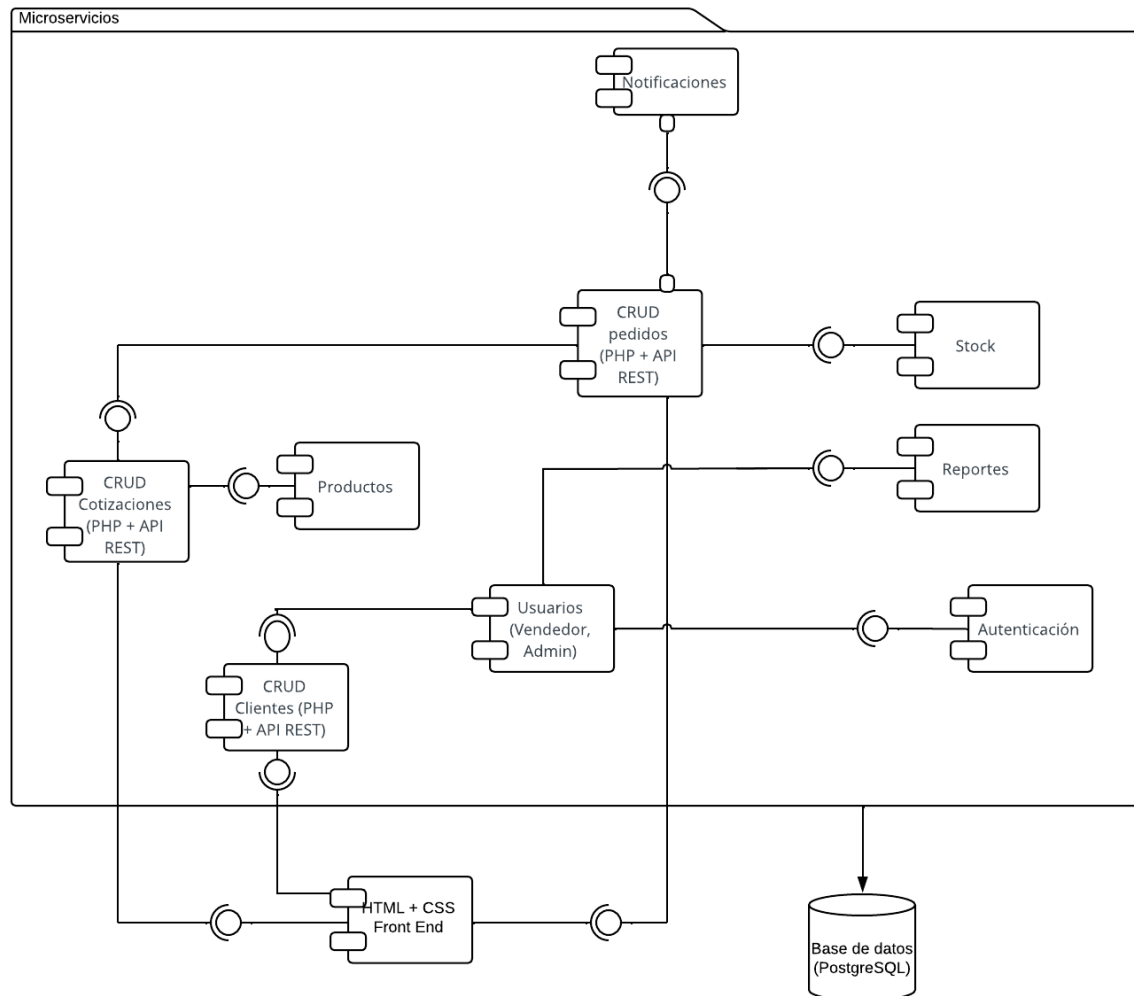
Productos: A medida que el catálogo de productos crezca, el servicio de productos debe ser capaz de manejar un mayor número de elementos.

Justificación:

La arquitectura de microservicios permite escalabilidad horizontal y vertical. Esto significa que, si la demanda aumenta, se pueden agregar más instancias de los servicios relacionados (como cotizaciones y pedidos) sin afectar a otros. Además, los servicios pueden escalar independientemente según su carga de trabajo específica, lo que garantiza una optimización

de recursos. Por ejemplo, en una temporada de alta demanda de cotizaciones o pedidos, el servicio de cotizaciones puede ser escalado sin afectar el servicio de los productos.

d. Diagrama de componentes y comunicación entre servicios

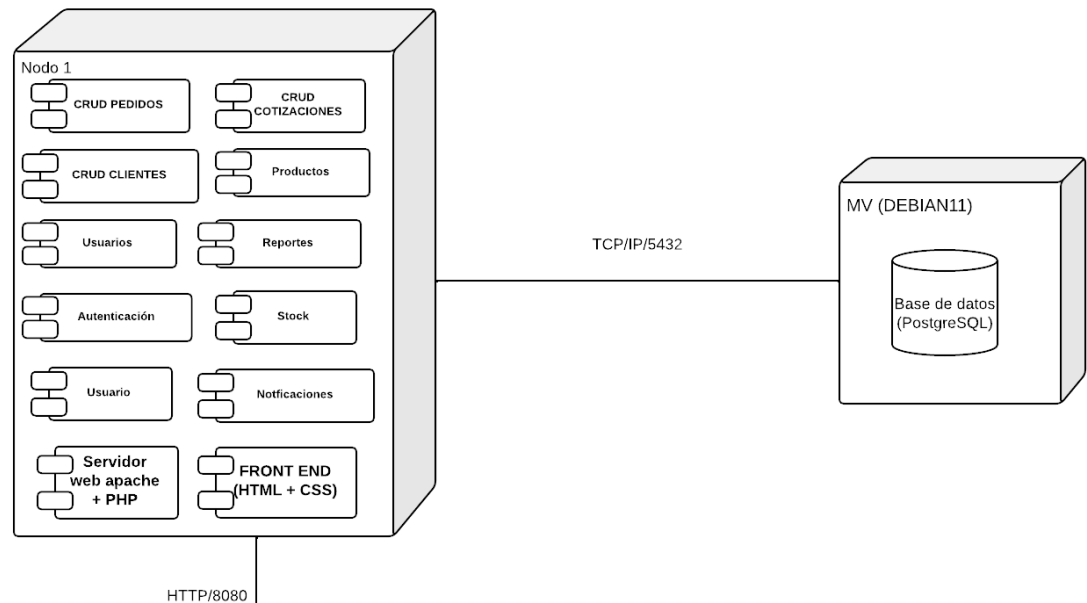


Comunicación entre servicios:

- El componente CRUD cotizaciones ofrece una interfaz que requiere el componente CRUD pedidos para realizar el pedido a partir de una cotización previa realizada.
- El componente Productos ofrece una interfaz para consultar los productos disponibles, y cotizaciones la requiere para ver los productos disponibles a cotizar.
- El componente CRUD cliente ofrece una interfaz que es requerida por el componente Usuarios, ya que el usuario vendedor es el encargado de realizar el proceso.
- El componente Notificaciones ofrece una interfaz que se requiere cuando se confirma un pedido realizado a partir de una cotización se envía una notificación a WhatsApp, por eso se abre un puerto en ese caso (por ser un servicio externo).
- El componente CRUD Pedidos requiere una interfaz de stock, ya que antes de confirmar un pedido se requiere validar el stock de los productos disponibles (elemento).

- El componente Reportes ofrece una interfaz para generar los reportes que pueden ser consultados por los dos usuarios que posee el sistema (Vendedor y Admin)
- El componente Reportes ofrece una interfaz para generar los reportes que pueden ser consultados por los dos usuarios que posee el sistema (Vendedor y Admin)
- Todos los microservicios se conectan a la base de datos Postgres

f. Diagrama de despliegue.



2. Diseño de los Microservicios. Por cada microservicio:

1. Microservicio de Autenticación (Auth)

a. Nombre y propósito.

Nombre: Autenticación (Auth)

Propósito: Gestionar la autenticación de los usuarios mediante JWT. Verificar las credenciales de los usuarios y generar un token de acceso que se utilizará para validar las acciones de la API.

b. Interfaces expuestas (endpoints REST)

Método	Endpoint	Descripción
POST	/co.edu.unbosque.auth/login	Permite el login del usuario mediante credenciales (usuario y contraseña). Retorna un token JWT para autorizar futuras solicitudes.

2. Microservicio de Clientes

a. Nombre y propósito

Nombre: Clientes

Propósito: Gestionar los datos de los clientes (naturales y empresas), incluyendo la creación, actualización, eliminación y consulta de clientes.

b. Interfaces expuestas (endpoints REST)

Método	Endpoint	Descripción
POST	/co.edu.unbosque.cliente	Crear un cliente nuevo.
GET	/co.edu.unbosque.cliente/{id}	Consultar a un cliente por su ID.
PUT	/co.edu.unbosque.cliente/{id}	Actualizar los datos de un cliente.
GET	/co.edu.unbosque.cliente	Listar todos los clientes.
DELETE	/co.edu.unbosque.cliente/{id}	Eliminar (lógicamente) un cliente por su ID.

3. Microservicio de Vendedores

a. Nombre y propósito

Nombre: Vendedores

Propósito: Gestionar los datos de los vendedores, principalmente para la consulta de todos los vendedores disponibles.

b. Interfaces expuestas (endpoints REST)

Método	Endpoint	Descripción
GET	/co.edu.unbosque.vendedor	Consultar a todos los vendedores disponibles.

4. Microservicio de Cotizaciones

a. Nombre y propósito

Nombre: Cotizaciones

Propósito: Gestionar la creación, consulta, modificación y eliminación de cotizaciones, así como la manipulación de los detalles de cada cotización.

b. Interfaces expuestas (endpoints REST)

Método	Endpoint	Descripción
POST	/co.edu.unbosque.cotizacion	Crear una nueva cotización.
GET	/co.edu.unbosque.cotizacion	Consultar todas las cotizaciones.

PUT	/co.edu.unbosque.cotizacion/{id}	Cambiar el estado de una cotización (aprobada, cancelada).
DELETE	/co.edu.unbosque.cotizacion/{id}	Eliminar una cotización.
PUT	/co.edu.unbosque.cotizacion/edit/{id}	Recalcular totales de una cotización.

5. Microservicio de Pedidos

a. Nombre y propósito

Nombre: Pedidos

Propósito: Gestionar los pedidos realizados por los clientes, desde la creación del pedido hasta su actualización o eliminación. Incluye la manipulación de los detalles del pedido y su estado.

b. Interfaces expuestas (endpoints REST)

Método	Endpoint	Descripción
POST	/co.edu.unbosque.pedido	Crear un nuevo pedido.
GET	/co.edu.unbosque.pedido	Consultar todos los pedidos.
PUT	/co.edu.unbosque.pedido/{id}	Cambiar el estado de un pedido (confirmado, cancelado).
DELETE	/co.edu.unbosque.pedido/{id}	Eliminar un pedido.
POST	/co.edu.unbosque.detallepedido	Crear un detalle de pedido.
GET	/co.edu.unbosque.detallepedido/{id}	Consultar los detalles de un pedido.
PUT	/co.edu.unbosque.detallepedido/{id}	Actualizar un detalle de pedido.
DELETE	/co.edu.unbosque.detallepedido/{id}	Eliminar un detalle de pedido.

6. Microservicio de Elementos (Productos)

a. Nombre y propósito

Nombre: Elementos (Productos)

Propósito: Gestionar los productos disponibles, permitiendo listar todos los productos.

b. Interfaces expuestas (endpoints REST)

Método	Endpoint	Descripción
GET	/co.edu.unbosque.elemento	Consultar todos los productos disponibles.

7. Microservicio de Auditoría

a. Nombre y propósito

Nombre: Auditoría

Propósito: Registrar y consultar todas las acciones realizadas en la aplicación (incluso accesos, cambios de estado, etc.), para permitir un control detallado de los eventos importantes.

b. Interfaces expuestas (endpoints REST)

Método	Endpoint	Descripción
GET	/co.edu.unbosque.auditoria	Consultar todos los registros de auditoría.

8. Microservicio de Notificaciones

a. Nombre y propósito

Nombre: Notificaciones

Propósito: Enviar notificaciones por correo electrónico o WhatsApp al cliente, tanto para cotizaciones como pedidos.

b. Interfaces expuestas (endpoints REST)

Método	Endpoint	Descripción
POST	/co.edu.unbosque.notificacion/cotizacion/{id}/email	Enviar una cotización por correo electrónico.
POST	/co.edu.unbosque.notificacion/cotizacion/{id}/whatsapp	Enviar una cotización por WhatsApp.
POST	/co.edu.unbosque.notificacion/pedido/{id}/email	Enviar un pedido por correo electrónico.
POST	/co.edu.unbosque.notificacion/pedido/{id}/whatsapp	Enviar un pedido por WhatsApp.

c. Modelo de datos (sin relaciones, sin índices, con relaciones, con índices)

Antes:

Con relaciones y llaves:

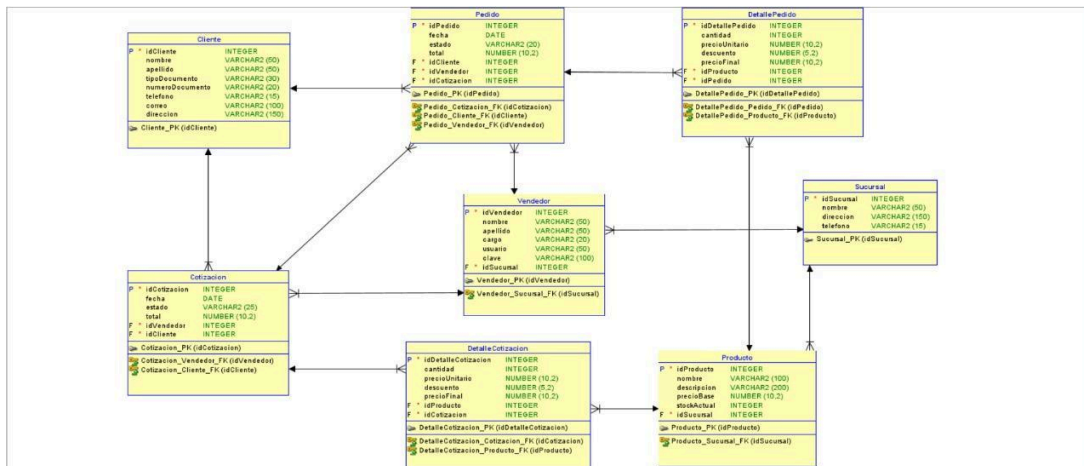
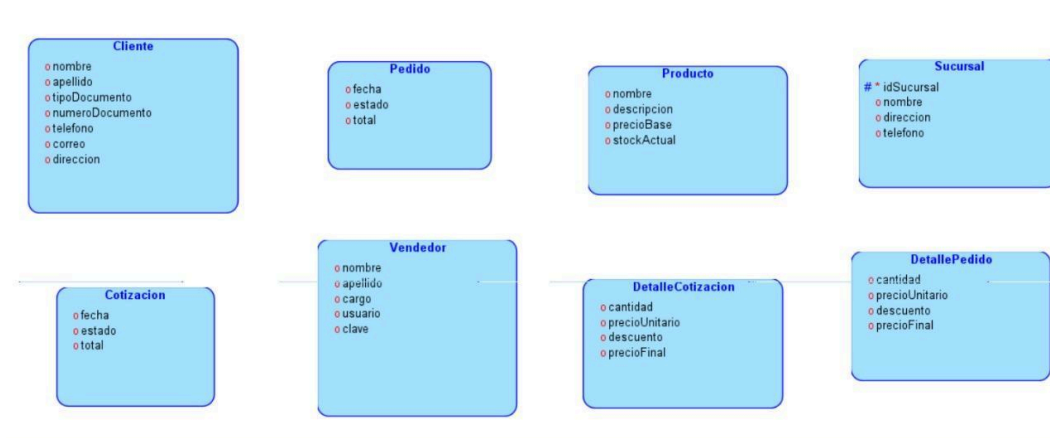


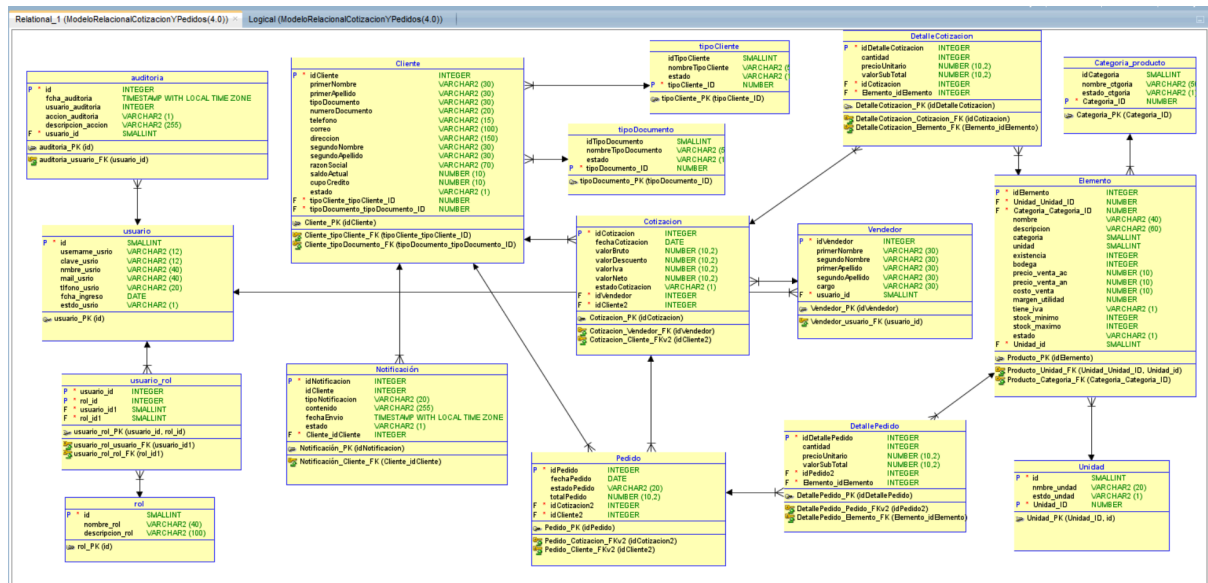
Fig 1.

Sin relaciones y sin llaves:

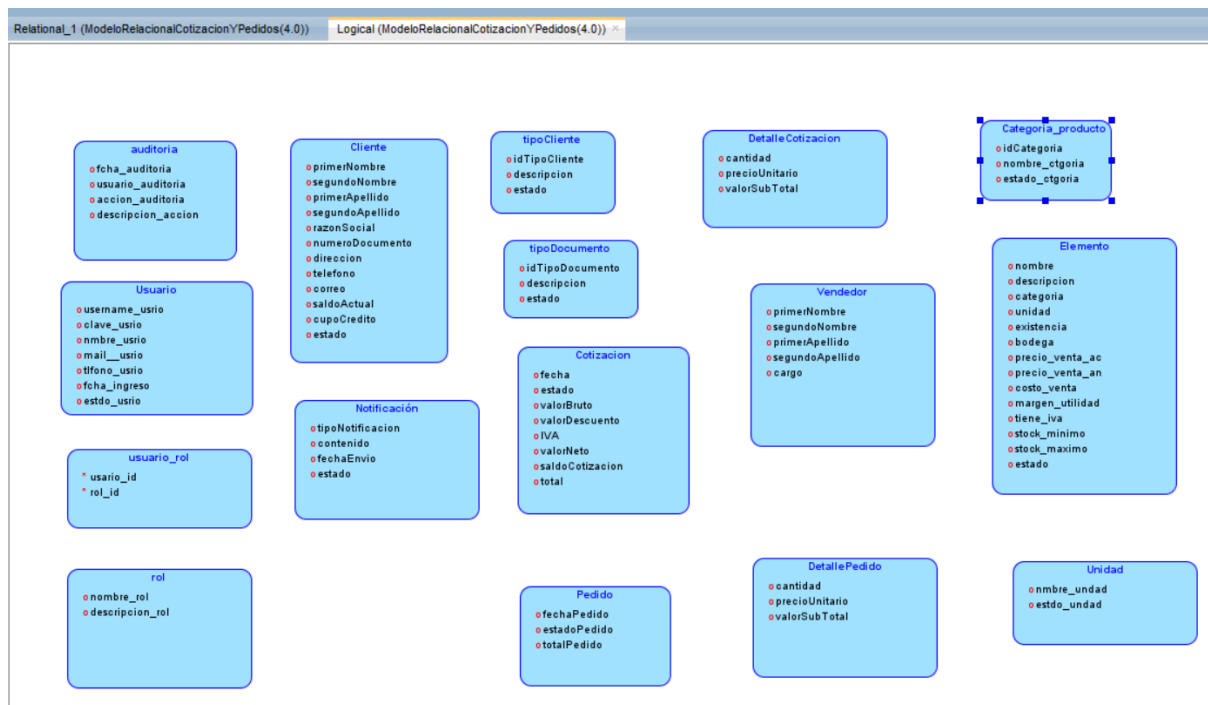


Después:

Con relaciones y llaves:



Con relaciones y llaves:



- ¿Qué se tomó del ScriptBD_v3?

1. Tabla Usuario

Tomado del script:

- id
- username_usuario
- clave_usuario

- nmbre_usrio
- mail_usrio
- tlfno_usrio
- fcha_ingrso
- estdo_usrio

Ajustes realizados:

- Solo se ajustó el atributo **estado_usrio** y se ajustó el valor posible a "**A**" (**Activo**) y "**I**" (**Inactivo**).

¿Por qué se mantuvo?

La tabla **usuario** es fundamental para la gestión de los usuarios dentro del sistema, como autenticación y asignación de roles. Este cambio solo se enfocó en permitir representar el estado del usuario (Activo o Inactivo). El resto de los atributos se mantienen igual porque son necesarios para controlar los usuarios dentro del sistema

2. Tabla Rol

Tomado del script:

- id
- nmbre_rol
- dscrpcion_rol

¿Por qué se mantuvo?

La tabla **rol** es obligatoria para la gestión de roles dentro del sistema, esta permitió separar correctamente las responsabilidades para los dos únicos roles del sistema (Vendedor y Administrador).

3. Tabla Usuario_Rol

Tomado del script:

- usuario_id
- rol_id

¿Por qué se mantuvo?

La tabla **usuario_rol** es esencial para gestionar la relación muchos a muchos entre usuarios y roles, lo cual permite asignar múltiples roles a un usuario. No se hizo ningún cambio porque esta tabla es perfectamente funcional tal como estaba en el script original.

4. Tabla Cliente

Tomado del script:

- id
- drccion_clnte
- correo_clnte
- tlfno_clnte
- sldoact_clnte
- cpocrdto_clnte
- estdo_clnte

¿Por qué se mantuvo y qué se modificó?

- Se eliminaron los atributos **nit_clnte**, **ctgoria_cliente**, **rprsnte_cliente**, **cntacto1_clnte**, **cntacto2_clnte**, **cmpras_mes**, **pagos_mes**, **condicion_pago**, ya que no son relevantes para la lógica del módulo cotizaciones y pedidos, por tal motivo solo se mantuvieron los necesarios.

5. Tabla Categoria_Producto

Tomado del script:

- id
- nombre_ctgria
- estdo_ctgria

¿Por qué se mantuvo?

La tabla **categoria_producto** es clave para clasificar los productos de la empresa y facilitar las búsquedas. No se hicieron cambios porque ya estaba adecuada para el propósito de organizar los productos en categorías.

6. Tabla Unidad

Tomado del script:

- id
- nombre_undad
- estdo_undad

¿Por qué se mantuvo?

La tabla **unidad** es de vital importancia para poder gestionar las unidades en que se venden los productos (por ejemplo, cajas, unidades, litros, etc.). La tabla no sufrió cambios, ya que su estructura es adecuada para gestionar las unidades de productos.

7. Tabla Elemento

Tomado del script:

- Id
- nombre
- dscrpcion_elemnto
- ctgria_elemnto
- und_elemnto
- exstncia_elemnto
- bdga_elemnto
- precio_venta_ac
- precio_venta_an
- costo_venta
- mrgen_utldad
- tiene_iva
- stock_minimo
- stock_maximo
- estdo_elmnto

¿Por qué se mantuvo?

La tabla **elemento** es fundamental para gestionar los productos que se venden. Se eliminarón los siguientes atributos: **sku_elemnto**, **lote_elemnto**, **prsntacion_elemnto**, **lbrtorio_elemnto**, **cntrolado_elemnto**. Esta desición se tomó debido a que no se consideran necesarios para el módulo, y como tal solo se requieren conocer los precios, estados y stock de los elementos (productos), el resto de los atributos son innecesarios.

8. Tabla Auditoría

Tomado del script:

- id
- fcha_audtria
- usrio_audtria
- accion_audtria

¿Por qué se mantuvo?

Simplemente se añadió un atributo **descripcion_accion** de tipo VARCHAR (255) para registrar una descripción más detallada de la acción realizada, como la inserción, actualización o eliminación de registros, facilitando la trazabilidad y auditoría de la actividad del sistema. De resto la tabla se mantuvo exactamente igual.

9. Tabla Cotización

Tomado del script:

- id
- id_cliente
- fecha_ctzcion
- vlór_brto
- vlór_dscto
- vlór_impsto (o IVA)
- vlór_nto
- estdo_ctzcion

¿Por qué se mantuvo?

- La tabla se mantuvo porque representa una la acción más importante del sistema, “las cotizaciones”. Por su parte, solo se eliminó el atributo **vldez_ctzcion** porque no era necesario para la lógica de la cotización. Este atributo representaba una fecha de validez, lo cual no resulta esencial.

IMPORTANTE: Para el descuento se estableció una regla solo para los meses de Noviembre (11) y Diciembre (12) en donde se aplica un descuento de 10 porciento para todos los pedidos confirmados hasta la fecha. Esto solo aplica para esa época del año, el resto no.

10. Tabla Detalle_Cotización

Tomado del script:

- id
- id_ctzcion
- id_elemnto
- cntdad_elemnto
- precio_untrio

¿Por qué se mantuvo?

- La tabla se mantuvo porque cada cotización debe tener un detalle en donde se especifiquen valores relevantes para el sistema y luego con estos, se recalculen los datos de la tabla Cotización. Por ejemplo, la cantidad de elementos y el precio unitario son necesarios para obtener el valor neto de la cotización. Se eliminaron los atributos **vlr_dscto** y **vlr_impsto**, debido a que se ya se manejan dentro de la tabla Cotización y no es necesario duplicarlos ya que estas tablas se encuentran relacionadas. Y solo se agregó el atributo **valorSubtotal**.

11. Tabla Pedido

Tomado del script:

- id
- fcha_pedido
- estdo_pedido

¿Por qué se mantuvo?

- La tabla “pedido” se mantuvo, debido a que a partir de una cotización siempre se debe poder generar un pedido, por tal motivo es obligatoria para el sistema. Se tomaron solo los atributos relevantes de la tabla **pedido** original (fecha, estado y total), y se eliminaron el resto, debido a que en el modelo relacional se estableció una relación entre la tabla cotización, y la tabla “pedido”, por tal motivo no era necesario duplicar datos como el IVA, valor descuento, etc.

12. Tabla Detalle_Pedido

Tomado del script:

- id
- cntdad_elemnto
- prcio_untrio

¿Por qué se mantuvo?

- Se eliminaron los atributos **vlr_dscto** y **vlr_impsto** y solo se agregó el atributo: **valorSubtotal**. De igual manera, como se decidió para el detalle de la cotización, en este caso tampoco resulta necesario manejar los valores de descuento, precio, iva, etc debido a que como un pedido proviene de una cotización previa, estos datos ya se obtienen.

d. Diccionario de datos.

1. Tabla: tipo_cliente

Descripción:

La tabla tipo_cliente define los tipos de clientes en el sistema, que pueden ser personas naturales o jurídicas (empresas). Esta tabla se usa para asociar a cada cliente con un tipo específico.

Relaciones:

Relación de uno a muchos con la tabla cliente. Un tipo de cliente puede estar asociado a varios clientes, pero cada cliente solo tiene un tipo.

Atributo	Tipo de Dato	Descripción
idTipoCliente	SERIAL	Clave primaria de la tabla. Identificador único del tipo de cliente.
nombreTipoCliente	VARCHAR (50)	Nombre del tipo de cliente (Ejemplo: "Persona Natural", "Empresa").
estado	VARCHAR (1)	Estado del tipo de cliente, puede ser "A" (Activo) o "I" (Inactivo).

2. Tabla: tipo_documento

Descripción:

La tabla tipo_documento almacena los diferentes tipos de documentos de identificación que los clientes pueden tener (por ejemplo, cédula, NIT, pasaporte).

Relaciones:

Relación de uno a muchos con la tabla cliente. Cada cliente debe tener un tipo de documento asociado.

Atributo	Tipo de Dato	Descripción
idTipoDocumento	SERIAL	Clave primaria de la tabla. Identificador único del tipo de documento.
nombreTipoDocumento	VARCHAR (50)	Nombre del tipo de documento (Ejemplo: "Cédula", "NIT", "Pasaporte").
estado	VARCHAR (1)	Estado del tipo de documento, puede ser "A" (Activo) o "I" (Inactivo).

3. Tabla: cliente

Descripción:

La tabla cliente almacena la información de los clientes. Contiene los datos relevantes de cada cliente, ya sea persona natural o jurídica, como su documento, nombre, contacto, etc.

Relaciones:

- Relación de muchos a uno con la tabla tipo_cliente. Cada cliente tiene un tipo asociado.
- Relación de muchos a uno con la tabla tipo_documento. Cada cliente tiene un tipo de documento.
- Relación de uno a muchos con la tabla cotizacion. Un cliente puede tener varias cotizaciones.
- Relación de uno a muchos con la tabla pedido. Un cliente puede tener varios pedidos.

Atributo	Tipo de Dato	Descripción
idCliente	SERIAL	Clave primaria de la tabla. Identificador único del cliente.
idTipoCliente	INT	Clave foránea a la tabla tipo_cliente . Tipo de cliente.
idTipoDocumento	INT	Clave foránea a la tabla tipo_documento . Tipo de documento del cliente.
numeroDocumento	VARCHAR (15)	Número de documento del cliente (Ejemplo: cédula, NIT, etc.).
primerNombre	VARCHAR (30)	Primer nombre del cliente (si es persona natural).
segundoNombre	VARCHAR (30)	Segundo nombre del cliente (si es persona natural).
primerApellido	VARCHAR (30)	Primer apellido del cliente (si es persona natural).
segundoApellido	VARCHAR (30)	Segundo apellido del cliente (si es persona natural).
razonSocial	VARCHAR (70)	Razón social del cliente (solo para empresas).
telefono	VARCHAR (15)	Número de teléfono del cliente.
correo	VARCHAR (100)	Correo electrónico del cliente.
direccion	VARCHAR (150)	Dirección del cliente.
saldoActual	NUMERIC (10,2)	Saldo actual del cliente en el sistema.
cupoCredito	NUMERIC (10,2)	Cupo de crédito asignado al cliente.
estado	VARCHAR (1)	Estado del cliente (Activo "A" o Inactivo "I").

4. Tabla: vendedor

Descripción:

La tabla “vendedor” almacena la información de los vendedores del sistema. Está relacionada con la tabla usuario.

Relaciones:

Relación de uno a uno con la tabla de usuario. Cada vendedor es un usuario del sistema.

Atributo	Tipo de Dato	Descripción
idVendedor	SERIAL	Clave primaria de la tabla. Identificador único del vendedor.
idUsuario	INT	Clave foránea a la tabla usuario . Relación con el usuario que representa al vendedor.
primerNombre	VARCHAR (30)	Primer nombre del vendedor.
segundoNombre	VARCHAR (30)	Segundo nombre del vendedor.
primerApellido	VARCHAR (30)	Primer apellido del vendedor.

segundoApellido	VARCHAR (30)	Segundo apellido del vendedor.
cargo	VARCHAR (100)	Cargo del vendedor en la empresa.

5. Tabla: cotizacion

Descripción:

La tabla cotizacion gestiona las cotizaciones de los clientes. Contiene los detalles de las cotizaciones realizadas, como los productos cotizados, los precios y los descuentos aplicados.

Relaciones:

Relación de muchos a uno con la tabla cliente. Cada cotización está asociada a un cliente.

Relación de muchos a uno con la tabla vendedor. Cada cotización es realizada por un vendedor.

Atributo	Tipo de Dato	Descripción
idCotizacion	SERIAL	Clave primaria de la tabla. Identificador único de la cotización.
idCliente	INT	Clave foránea a la tabla cliente . Cliente asociado a la cotización.
idVendedor	INT	Clave foránea a la tabla vendedor . Vendedor que realiza la cotización.
fechaCotizacion	DATE	Fecha en que se realiza la cotización.
valorBruto	NUMERIC (10,2)	Valor bruto total de la cotización (sin descuentos ni impuestos).
valorDescuento	NUMERIC (10,2)	Valor del descuento aplicado a la cotización.
valorIVA	NUMERIC (10,2)	Valor del IVA aplicado a la cotización.
valorNeto	NUMERIC (10,2)	Valor neto de la cotización (descuento e IVA aplicados).
estadoCotizacion	VARCHAR (1)	Estado de la cotización (P=Pendiente, A=Aprobada, C=Cancelada).

6. Tabla: detalle_cotizacion

Descripción:

La tabla detalle_cotizacion almacena los productos específicos dentro de una cotización, incluyendo las cantidades, precios unitarios y subtotales.

Relaciones:

Relación de muchos a uno con la tabla cotizacion. Cada detalle está asociado a una cotización específica.

Relación de muchos a uno con la tabla elemento. Cada detalle está asociado a un producto.

Atributo	Tipo de Dato	Descripción
idDetalleCotizacion	SERIAL	Clave primaria de la tabla. Identificador único del detalle de la cotización.
idCotizacion	INT	Clave foránea a la tabla cotizacion . Cotización a la que pertenece el detalle.
idProducto	INT	Clave foránea a la tabla elemento . Producto dentro de la cotización.
cantidad	INT	Cantidad del producto en la cotización.
precioUnitario	NUMERIC (10,2)	Precio unitario del producto en la cotización.
valorSubtotal	NUMERIC (10,2)	Subtotal por producto (cantidad * precio unitario).

7. Tabla: pedido

Descripción:

La tabla “pedido” gestiona los pedidos realizados por los clientes. Cada pedido está relacionado con una cotización previa y contiene el estado del pedido, su total y la fecha en que se realizó.

Relaciones:

Relación de muchos a uno con la tabla cliente. Cada pedido está asociado a un cliente.

Relación de muchos a uno con la tabla cotizacion. Cada pedido se origina de una cotización específica.

Atributo	Tipo de Dato	Descripción
idPedido	SERIAL	Clave primaria de la tabla. Identificador único del pedido.
idCliente	INT	Clave foránea a la tabla cliente . Cliente que realizó el pedido.
idCotizacion	INT	Clave foránea a la tabla cotizacion . Cotización asociada al pedido.
fechaPedido	DATE	Fecha en que se realizó el pedido.
estadoPedido	VARCHAR (20)	Estado del pedido (Pendiente, Confirmado, Cancelado).
totalPedido	NUMERIC (10,2)	Total del pedido (valor final con impuestos y descuentos).

8. Tabla: detalle_pedido

Descripción:

La tabla detalle_pedido almacena los detalles de los productos dentro de un pedido. Similar a la tabla de detalle de cotización, almacena las cantidades, precios y subtotales por producto.

Relaciones:

Relación de muchos a uno con la tabla pedido. Cada detalle está asociado a un pedido.

Relación de muchos a uno con la tabla elemento. Cada detalle corresponde a un producto específico.

Atributo	Tipo de Dato	Descripción
idDetallePedido	SERIAL	Clave primaria de la tabla. Identificador único del detalle del pedido.
idPedido	INT	Clave foránea a la tabla pedido . Pedido asociado al detalle.
idProducto	INT	Clave foránea a la tabla elemento . Producto dentro del pedido.
cantidad	INT	Cantidad del producto en el pedido.
precioUnitario	NUMERIC (10,2)	Precio unitario del producto en el pedido.
valorSubtotal	NUMERIC (10,2)	Subtotal por producto (cantidad * precio unitario).

9. Tabla: elemento

Descripción:

La tabla elemento almacena los productos disponibles en el sistema. Cada producto tiene información sobre su categoría, unidad, precio, stock, entre otros.

Relaciones:

Relación de muchos a uno con la tabla unidad. Cada producto tiene una unidad de medida asociada.

Relación de muchos a uno con la tabla categoria_producto. Cada producto pertenece a una categoría

Atributo	Tipo de Dato	Descripción
idElemento	SERIAL	Clave primaria de la tabla. Identificador único del producto.
Unidad_Unidad_ID	INT	Clave foránea a la tabla unidad . Unidad de medida del producto.

Categoria_Categoria_ID	INT	Clave foránea a la tabla categoria_producto . Categoría del producto.
nombre	VARCHAR (40)	Nombre del producto.
descripcion	VARCHAR (60)	Descripción breve del producto.
categoria	SMALLINT	Relación con la categoría del producto.
unidad	SMALLINT	Relación con la unidad del producto.
existencia	INT	Stock actual del producto en el inventario.
bodega	INT	Número de bodega donde se almacena el producto.
precio_venta_ac	NUMERIC (10,2)	Precio de venta actual del producto.
precio_venta_an	NUMERIC (10,2)	Precio de venta anterior del producto.
costo_venta	NUMERIC (10,2)	Costo de venta del producto.
margen_utilidad	NUMERIC	Margen de utilidad calculado sobre el producto.
tiene_iva	VARCHAR (1)	Indica si el producto tiene IVA (Y=Sí, N=No).
stock_minimo	INT	Stock mínimo permitido para el producto.
stock_maximo	INT	Stock máximo permitido para el producto.
estado	VARCHAR (1)	Estado del producto (Activo "A" o Inactivo "I").

10. Tabla: categoria_producto

Descripción:

La tabla categoria_producto almacena las categorías de productos disponibles en el sistema. Cada producto pertenece a una categoría para facilitar su organización y clasificación.

Relaciones:

Relación de uno a muchos con la tabla elemento. Una categoría puede contener muchos productos, pero cada producto tiene solo una categoría.

Atributo	Tipo de Dato	Descripción
id	SERIAL	Clave primaria de la tabla. Identificador único de la categoría.
nombreCtgria	VARCHAR (50)	Nombre de la categoría del producto (Ejemplo: "Medicamentos", "Cosméticos").
estadoCtgria	VARCHAR (1)	Estado de la categoría (Activo "A" o Inactivo "I").

11. Tabla: unidad

Descripción:

La tabla unidad almacena las unidades de medida disponibles para los productos, como unidades, cajas, litros, etc.

Relaciones:

Relación de uno a muchos con la tabla elemento. Una unidad puede estar asociada a varios productos, pero cada producto tiene una unidad.

Atributo	Tipo de Dato	Descripción
id	SERIAL	Clave primaria de la tabla. Identificador único de la unidad.
nombreUnidad	VARCHAR (20)	Nombre de la unidad de medida (Ejemplo: "Caja", "Litro", "Unidad").
estadoUnidad	VARCHAR (1)	Estado de la unidad (Activo "A" o Inactivo "I").

12. Tabla: usuario

Descripción:

La tabla usuario almacena los usuarios del sistema. Los usuarios pueden ser clientes, vendedores, administradores, etc., y tienen acceso al sistema a través de credenciales.

Relaciones:

Relación de uno a uno con la tabla vendedor. Un usuario puede ser un vendedor, pero un vendedor es un usuario.

Atributo	Tipo de Dato	Descripción
id	SERIAL	Clave primaria de la tabla. Identificador único del usuario.
username_usuario	VARCHAR (12)	Nombre de usuario único para acceder al sistema.
clave_usuario	VARCHAR (12)	Contraseña del usuario para el acceso.
nombre_usuario	VARCHAR (40)	Nombre completo del usuario.
mail_usuario	VARCHAR (40)	Correo electrónico del usuario.
tlfno_usuario	VARCHAR (20)	Teléfono del usuario.
fcha_ingreso	DATE	Fecha de ingreso del usuario al sistema.
estado_usuario	VARCHAR (1)	Estado del usuario (Activo "A" o Inactivo "I").

13. Tabla: rol

Descripción:

La tabla rol define los roles que un usuario puede tener en el sistema, como administrador, vendedor, etc.

Relaciones:

Relación de muchos a muchos con la tabla usuario a través de la tabla usuario_rol.

Atributo	Tipo de Dato	Descripción
id	SERIAL	Clave primaria de la tabla. Identificador único del rol.
nombre_rol	VARCHAR (40)	Nombre del rol (Ejemplo: "Administrador", "Vendedor").
descripcion_rol	VARCHAR (100)	Descripción detallada del rol.

14. Tabla: usuario_rol

Descripción:

La tabla usuario_rol gestiona la relación muchos a muchos entre los usuarios y los roles que tienen asignados.

Relaciones:

Relación de muchos a muchos con la tabla usuario y rol.

Atributo	Tipo de Dato	Descripción
usuario_id	INT	Clave foránea a la tabla usuario . Relaciona el usuario con el rol.
rol_id	INT	Clave foránea a la tabla rol . Relaciona el rol con el usuario.

15. Tabla: auditoria

Descripción:

La tabla auditoría almacena el historial de acciones realizadas por los usuarios en el sistema, como inserciones, actualizaciones o eliminaciones.

Relaciones:

Relación de muchos a uno con la tabla usuario. Cada acción registrada está asociada a un usuario que la realizó.

Atributo	Tipo de Dato	Descripción
id	SERIAL	Clave primaria de la tabla. Identificador único de la auditoría.

fecha_auditoria	TIMESTAMP	Fecha y hora de la acción realizada.
usuario_auditoria	INT	Clave foránea a la tabla usuario . Usuario que realizó la acción.
accion_auditoria	VARCHAR (1)	Tipo de acción realizada (I=Insertar, U=Actualizar, D=Eliminar).
descripcion_accion	VARCHAR (255)	Descripción detallada de la acción realizada.

16. Tabla: notificacion

Descripción:

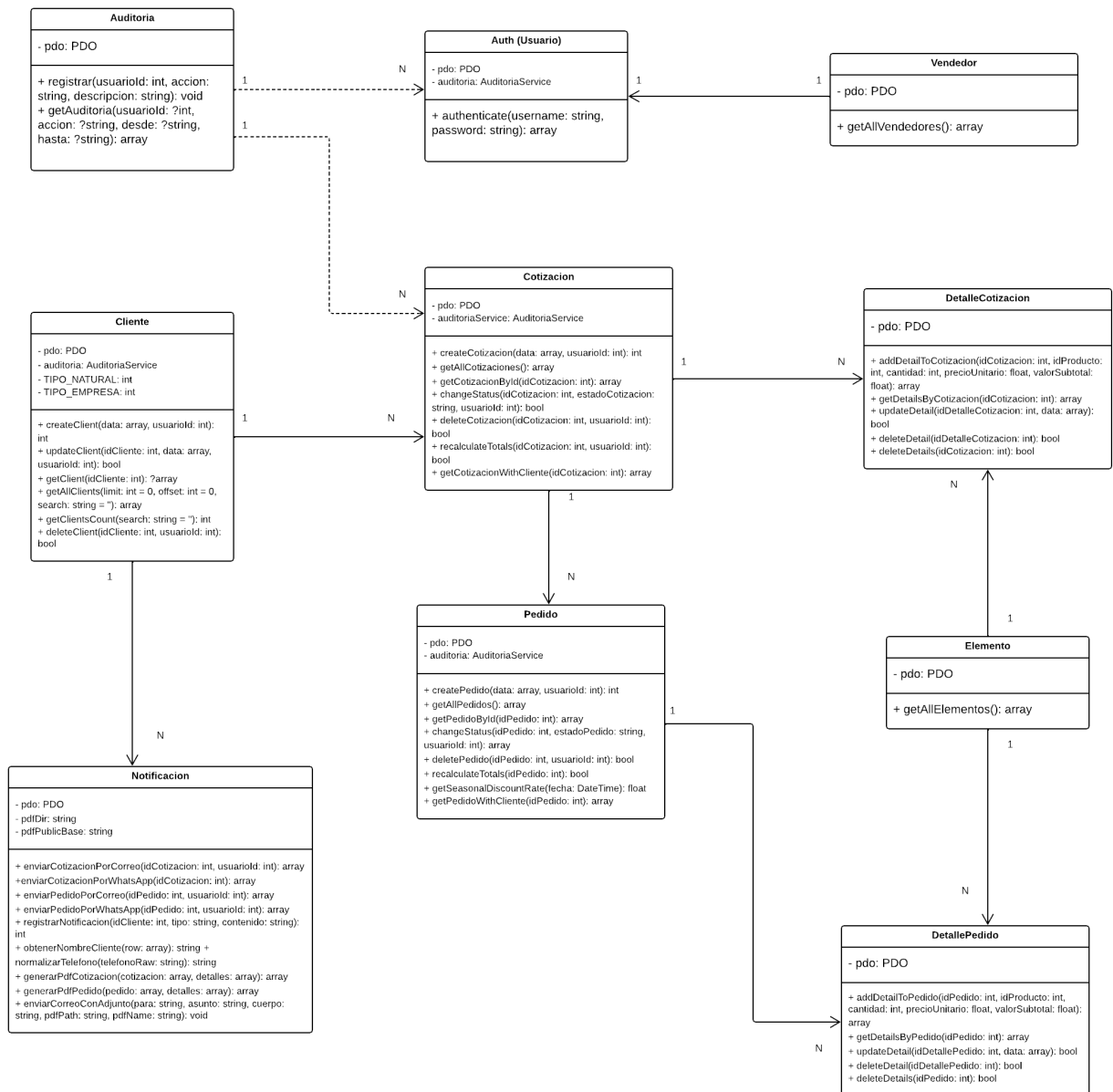
La tabla notificacion almacena las notificaciones que se envían a los clientes, ya sea por correo electrónico o WhatsApp.

Relaciones:

Relación de muchos a uno con la tabla cliente. Cada notificación está asociada a un cliente.

Atributo	Tipo de Dato	Descripción
idNotificacion	SERIAL	Clave primaria de la tabla. Identificador único de la notificación.
idCliente	INT	Clave foránea a la tabla cliente . Cliente que recibe la notificación.
tipoNotificacion	VARCHAR (20)	Tipo de notificación (Ejemplo: "Cotización", "Pedido").
contenido	VARCHAR (255)	Contenido de la notificación.
fechaEnvio	TIMESTAMP	Fecha y hora de envío de la notificación.
estado	VARCHAR (1)	Estado de la notificación (E=Enviado, P=Pendiente).

e. Diagrama de clases (si aplica).



Justificación del uso de PDO:

El uso de PDO (PHP Data Objects) en este proyecto se justifica debido a su seguridad, flexibilidad y compatibilidad con múltiples bases de datos. Al utilizar PDO, se asegura una protección robusta contra ataques de inyección SQL mediante el uso de prepared statements, lo que previene la manipulación maliciosa de las consultas. Además, PDO ofrece una interfaz consistente para trabajar con diferentes motores de bases de datos, facilitando la migración o el cambio de base de datos si fuera necesario. También, su soporte para el manejo de transacciones y la gestión de errores mejora la fiabilidad y consistencia del sistema.

¿Cómo lo implementamos en el proyecto?

En el proyecto, hemos implementado PDO en todos los servicios que interactúan con la base de datos, como la creación, actualización y eliminación de registros en tablas como Cliente, Cotizacion, Pedido, entre otras. Cada vez que realizamos una consulta, utilizamos prepared statements para prevenir ataques de inyección SQL, y además gestionamos las transacciones para asegurar que las operaciones se realicen correctamente. Las consultas se gestionan en funciones dentro de cada service, como CotizacionService o ClienteService, utilizando PDO para conectar y ejecutar las acciones requeridas sobre la base de datos, asegurando que los datos se manipulen de manera segura y eficiente.

f. Dependencias externas (bases de datos, otros servicios).

1. Dependencia con la Base de Datos

La base de datos es el núcleo del sistema, donde se almacenan todas las cotizaciones, pedidos, productos, clientes, detalles y otros datos clave que permiten el funcionamiento del módulo de Cotizaciones y Pedidos. Para gestionar la interacción entre los servicios y la base de datos, se utiliza PostgreSQL como motor de bases de datos relacional, implementado sobre una máquina virtual con Debian 11. La comunicación entre los servicios y la base de datos se realiza mediante el uso de PDO (PHP Data Objects), asegurando la flexibilidad y la seguridad en la ejecución de consultas SQL.

Dependencia clave:

Conexión a la base de datos a través de PDO, lo que permite realizar consultas SQL para insertar, actualizar, eliminar y recuperar datos.

Se hace uso de consultas SELECT, INSERT, UPDATE y DELETE en las tablas correspondientes.

Tablas relacionadas:

Tabla de Cotización: Almacena y recupera la información de las cotizaciones creadas por los usuarios.

Tabla de Pedido: Gestiona los pedidos asociados a las cotizaciones y su información relacionada.

Tabla de Cliente: Se utiliza para obtener los datos del cliente asociado con las cotizaciones y los pedidos.

Tabla de Elementos/Productos: Gestiona los productos que se incluyen en las cotizaciones y pedidos, incluyendo precios y cantidades.

Tablas de Detalles: Las tablas detalle_cotizacion y detalle_pedido gestionan la relación entre productos y cotizaciones/pedidos, permitiendo el almacenamiento de los detalles asociados a cada transacción.

Relaciones y dependencias específicas:

Los servicios de Cotización y Pedido dependen directamente de las tablas mencionadas para recuperar, insertar, actualizar o eliminar los datos relacionados con las cotizaciones, pedidos, clientes, productos y detalles.

La tabla de Auditoría: Todos los servicios de Cotización y Pedido también dependen de la tabla de auditoría para registrar las acciones realizadas sobre cotizaciones y pedidos, con fines de seguimiento e historial.

2. Dependencia con otros Microservicios (teóricamente)

Aunque el enfoque principal del proyecto es el módulo de Cotizaciones y Pedidos, dado que se trata de una arquitectura de microservicios, existen dependencias teóricas con otros servicios del sistema, los cuales podrían ser implementados en el futuro para complementar las funcionalidades del módulo trabajado. Las dependencias directas que se podrían tener con otros microservicios son:

Servicio de Facturación:

Dependencia: Cuando una cotización se convierte en un pedido confirmado, es probable que el módulo de Facturación deba generar la factura correspondiente. El servicio de Cotizaciones y Pedidos puede generar eventos o enviar solicitudes al servicio de Facturación para procesar la facturación después de la confirmación del pedido.

Servicio de Inventarios:

Dependencia: Los detalles de un pedido pueden involucrar la actualización de existencias de productos en el Inventario. Cuando se confirma un pedido, los servicios de Cotización y Pedido deben comunicarse con el servicio de Inventarios para ajustar las existencias de los productos relacionados con el pedido.

Servicio de Proveedores:

Dependencia: Si un pedido involucra productos de proveedores específicos, el servicio de Proveedores puede ser consultado para obtener información sobre la disponibilidad de productos o los precios actualizados. Los servicios del módulo pueden necesitar hacer consultas al servicio de Proveedores para verificar la disponibilidad y el precio de los productos incluidos en las cotizaciones y pedidos.

Servicio de Cartera:

Dependencia: Una vez que un pedido ha sido confirmado, el servicio de Cartera debería gestionar el proceso de pago, recibiendo los pagos de los clientes y registrando las transacciones. Los servicios de Pedido pueden depender del servicio de Cartera para procesar pagos y gestionar el estado financiero relacionado con el pedido.

3. Código fuente funcional.

a. El código fuente deberá entregarse mediante una USB o repositorio en la nube.

b. Código fuente de cada microservicio.

El código fuente del proyecto y de cada microservicio se entregará por medio de una USB.

c. API REST documentada con Swagger/Open API.

Para la documentación de la API se utilizó la librería swagger-php (zircote/swagger-php versión ^3.0), que permite generar especificaciones OpenAPI directamente desde anotaciones en el código.

Cada controlador de la aplicación fue documentado mediante anotaciones `@OA\*` en los métodos que exponen endpoints REST, describiendo rutas, parámetros, cuerpos de petición, respuestas y códigos de estado.

Posteriormente, se generó un archivo swagger.json mediante el comando de consola provisto por la librería, y finalmente se integró Swagger UI para visualizar la documentación completa de forma amigable desde el navegador.

El enlace de acceso es el siguiente: <http://localhost/proyectoFinalIS2/dist/>

Aquí se puede evidenciar lo siguiente:

localhost/proyectoFinalIS2/dist/

Swagger

/proyectoFinalIS2/swagger.json

Explore

API Proyecto Final IS2

1.0.0

QA 3.0

Documentación completa de la API del proyecto final

Contact the developer

Auditoría

GET/co.edu.unbosque.auditoriaObtiene todos los registros de auditoría

Autenticación

POST/co.edu.unbosque.auth/loginRealiza login de usuario

Cientes

GET/co.edu.unbosque.clienteObtiene todos los clientes

POST/co.edu.unbosque.clienteCrea uno o varios clientes

GET/co.edu.unbosque.cliente/tiposdocumentosObtiene todos los tipos de documentos activos

GET/co.edu.unbosque.cliente/{id}Obtiene un cliente por su ID

PUT/co.edu.unbosque.cliente/{id}Actualiza los datos de un cliente

DELETE/co.edu.unbosque.cliente/{id}Elimina (lógicamente) un cliente

Cotizaciones

GET/co.edu.unbosque.cotizacionObtiene todas las cotizaciones

POST/co.edu.unbosque.cotizacionCrea una nueva cotización

POST/co.edu.unbosque.detallecotizacionAgrega un detalle a una cotización

Cotizaciones

GET/co.edu.unbosque.cotizacionObtiene todas las cotizaciones

POST/co.edu.unbosque.cotizacionCrea una nueva cotización

POST/co.edu.unbosque.detallecotizacionAgrega un detalle a una cotización

PUT/co.edu.unbosque.cotizacion/{id}Cambia el estado de una cotización

DELETE/co.edu.unbosque.cotizacion/{id}Elimina una cotización y sus detalles

PUT/co.edu.unbosque.cotizacion/edit/{id}Recalcula los totales de una cotización

GET/co.edu.unbosque.detallecotizacion/{id}Obtiene los detalles de una cotización

PUT/co.edu.unbosque.detallecotizacion/{id}Actualiza un detalle de cotización

DELETE/co.edu.unbosque.detallecotizacion/{id}Elimina un detalle de cotización

Productos

GET/co.edu.unbosque.elementoObtiene todos los productos (elementos)

Pedidos

GET/co.edu.unbosque.pedidoObtiene todos los pedidos

POST/co.edu.unbosque.pedidoCrea un nuevo pedido (solo cabecera)

PUT/co.edu.unbosque.pedido/{id}Cambia el estado de un pedido

DELETE/co.edu.unbosque.pedido/{id}Elimina un pedido y sus detalles

Vendedores

GET/co.edu.unbosque.vendedorListar todos los vendedores

Notificaciones			^
POST	/co.edu.unbosque.notificacion/cotizacion/{id}/email	Envía una cotización por correo electrónico	▼
POST	/co.edu.unbosque.notificacion/cotizacion/{id}/whatsapp	Genera un enlace de WhatsApp para una cotización aprobada	▼
POST	/co.edu.unbosque.notificacion/pedido/{id}/email	Envía un pedido confirmado por correo electrónico	▼
POST	/co.edu.unbosque.notificacion/pedido/{id}/whatsapp	Genera un enlace de WhatsApp para un pedido confirmado	▼
Pedidos - Detalles			^
POST	/co.edu.unbosque.detallepedido	Agrega un detalle a un pedido	▼
GET	/co.edu.unbosque.detallepedido/{id}	Obtiene los detalles de un pedido	▼
PUT	/co.edu.unbosque.detallepedido/{id}	Actualiza un detalle de pedido	▼
DELETE	/co.edu.unbosque.detallepedido/{id}	Elimina un detalle de pedido	▼

d. Informe de pruebas unitarias y de integración (si aplica). Puede utilizar herramientas como: Junit, Postman, etc.

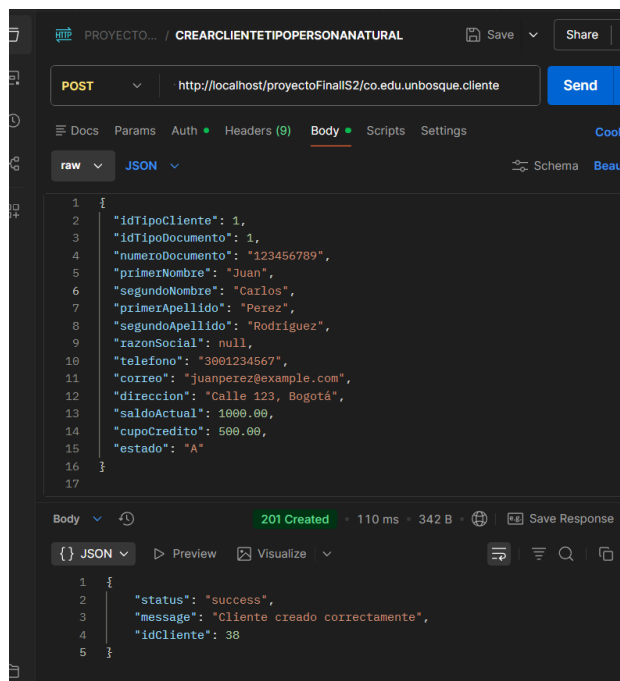
Para las pruebas se hizo uso de Postman:

1. Crear un cliente tipo persona natural

Método HTTP: POST

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.cliente>

Cuerpo de la solicitud (JSON) y resultado:

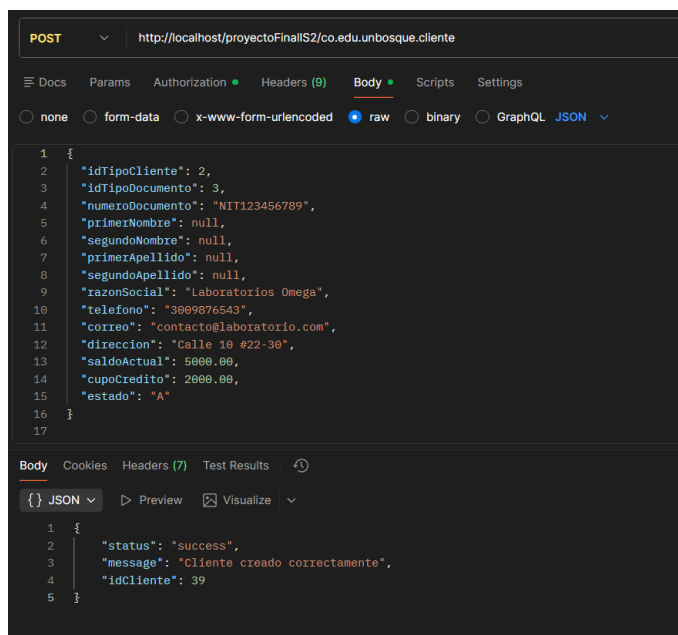


2. Crear un cliente tipo empresa

Método HTTP: POST

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.cliente>

Cuerpo de la solicitud (JSON) y resultado:



The screenshot shows a REST client interface with a POST request to `http://localhost/proyectoFinalIS2/co.edu.unbosque.cliente`. The request body is a JSON object with the following fields: `idTipoCliente` (2), `idTipoDocumento` (3), `numeroDocumento` ("NIT123456789"), `primerNombre` (null), `segundoNombre` (null), `primerApellido` (null), `segundoApellido` (null), `razonSocial` ("Laboratorios Omega"), `telefono` ("3009876543"), `correo` ("contacto@laboratorio.com"), `direccion` ("Calle 10 #22-30"), `saldoActual` (5000.00), `cupoCredito` (2000.00), and `estado` ("A"). The response is a JSON object with `status` ("success"), `message` ("Cliente creado correctamente"), and `idCliente` (39).

```
1 {
2   "idTipoCliente": 2,
3   "idTipoDocumento": 3,
4   "numeroDocumento": "NIT123456789",
5   "primerNombre": null,
6   "segundoNombre": null,
7   "primerApellido": null,
8   "segundoApellido": null,
9   "razonSocial": "Laboratorios Omega",
10  "telefono": "3009876543",
11  "correo": "contacto@laboratorio.com",
12  "direccion": "Calle 10 #22-30",
13  "saldoActual": 5000.00,
14  "cupoCredito": 2000.00,
15  "estado": "A"
16 }
17
```

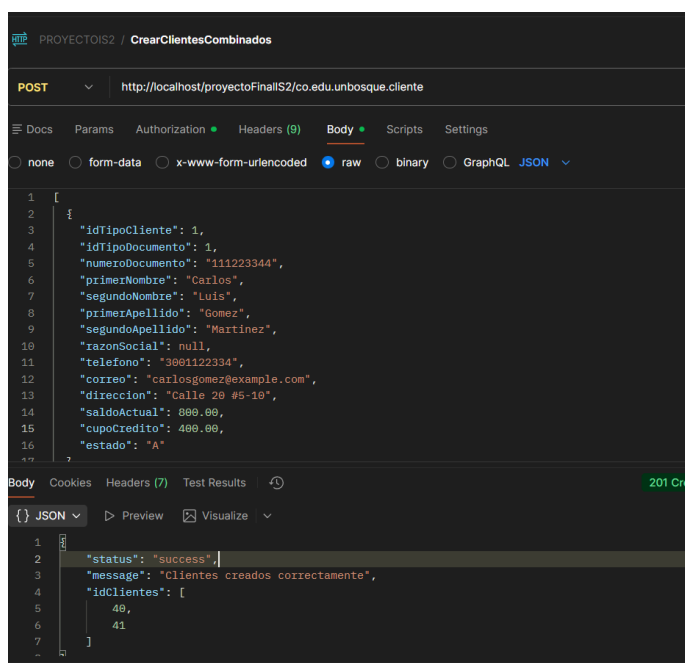
```
1 {
2   "status": "success",
3   "message": "Cliente creado correctamente",
4   "idCliente": 39
5 }
```

3. Crear 2 clientes combinados (naturales y empresas)

Método HTTP: POST

Ruta: `http://localhost/proyectoFinalIS2/co.edu.unbosque.cliente`

Cuerpo de la solicitud (JSON):



The screenshot shows a REST client interface with a POST request to `http://localhost/proyectoFinalIS2/co.edu.unbosque.cliente`. The request body is a JSON object with the following fields: `idTipoCliente` (1), `idTipoDocumento` (1), `numeroDocumento` ("111223344"), `primerNombre` ("Carlos"), `segundoNombre` ("Luis"), `primerApellido` ("Gomez"), `segundoApellido` ("Martinez"), `razonSocial` (null), `telefono` ("3001122334"), `correo` ("carlosgomez@example.com"), `direccion` ("Calle 20 #5-10"), `saldoActual` (800.00), `cupoCredito` (400.00), and `estado` ("A"). The response is a JSON object with `status` ("success"), `message` ("Clientes creados correctamente"), and `idClientes` ([40, 41]).

```
1 [
2   {
3     "idTipoCliente": 1,
4     "idTipoDocumento": 1,
5     "numeroDocumento": "111223344",
6     "primerNombre": "Carlos",
7     "segundoNombre": "Luis",
8     "primerApellido": "Gomez",
9     "segundoApellido": "Martinez",
10    "razonSocial": null,
11    "telefono": "3001122334",
12    "correo": "carlosgomez@example.com",
13    "direccion": "Calle 20 #5-10",
14    "saldoActual": 800.00,
15    "cupoCredito": 400.00,
16    "estado": "A"
17  },
18  {
19    "idTipoCliente": 1,
20    "idTipoDocumento": 1,
21    "numeroDocumento": "111223344",
22    "primerNombre": "Carlos",
23    "segundoNombre": "Luis",
24    "primerApellido": "Gomez",
25    "segundoApellido": "Martinez",
26    "razonSocial": null,
27    "telefono": "3001122334",
28    "correo": "carlosgomez@example.com",
29    "direccion": "Calle 20 #5-10",
30    "saldoActual": 800.00,
31    "cupoCredito": 400.00,
32    "estado": "A"
33  }
34 ]
```

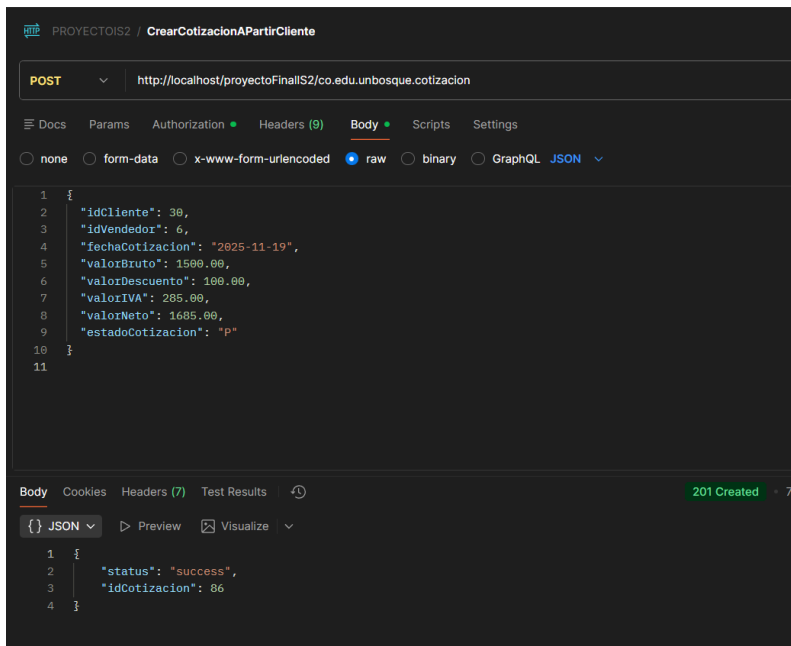
```
1 {
2   "status": "success",
3   "message": "Clientes creados correctamente",
4   "idClientes": [
5     40,
6     41
7   ]
8 }
```

4. Crear una cotización a partir del ID de un cliente

Método HTTP: POST

Ruta: `http://localhost/proyectoFinalIS2/co.edu.unbosque.cotizacion`

Cuerpo de la solicitud (JSON) y resultado:

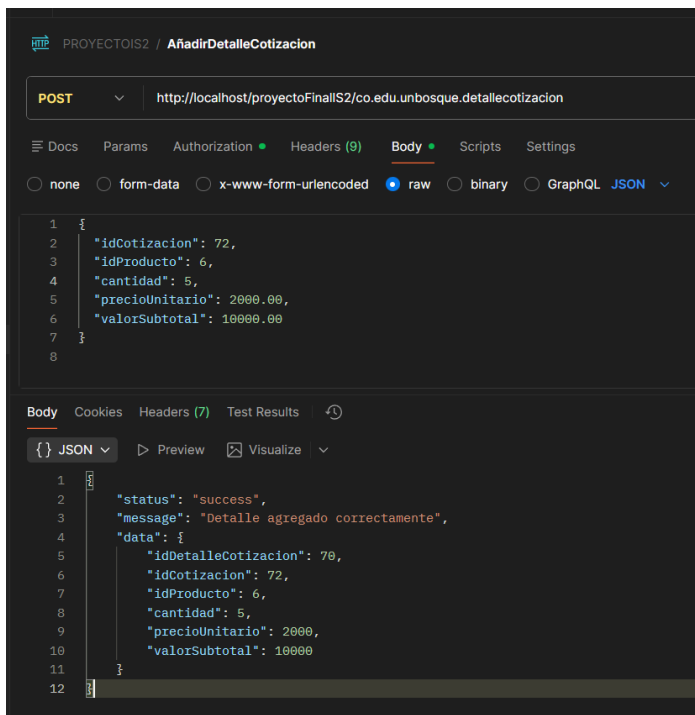


5. Añadir el detalle a una cotización

Método HTTP: POST

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.detallecotizacion>

Cuerpo de la solicitud (JSON):

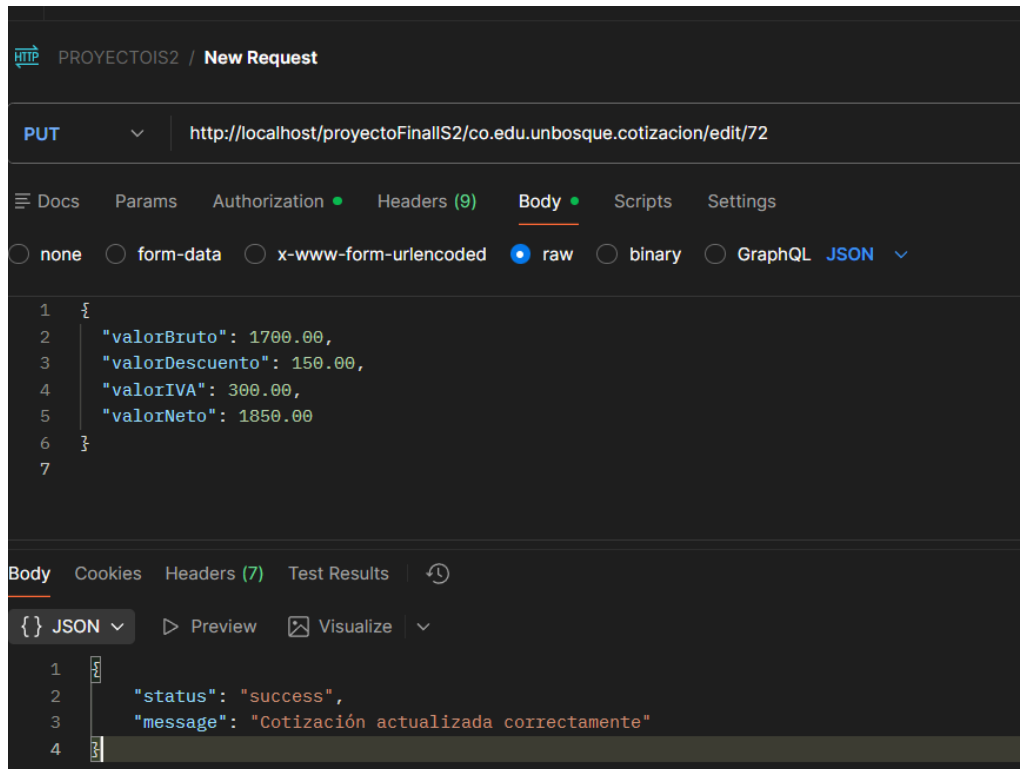


6. Editar una cotización

Método HTTP: PUT

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.cotizacion/edit/72>

Cuerpo de la solicitud (JSON):



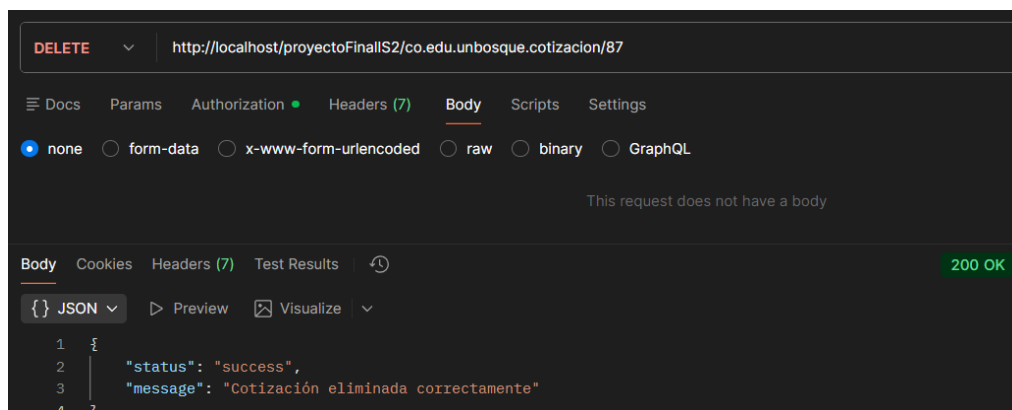
e. Configuración de CORS y seguridad básica (si aplica).

7. Eliminar una cotización

Método HTTP: DELETE

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.cotizacion/72>

Cuerpo de la solicitud: No es necesario, solo la solicitud DELETE.

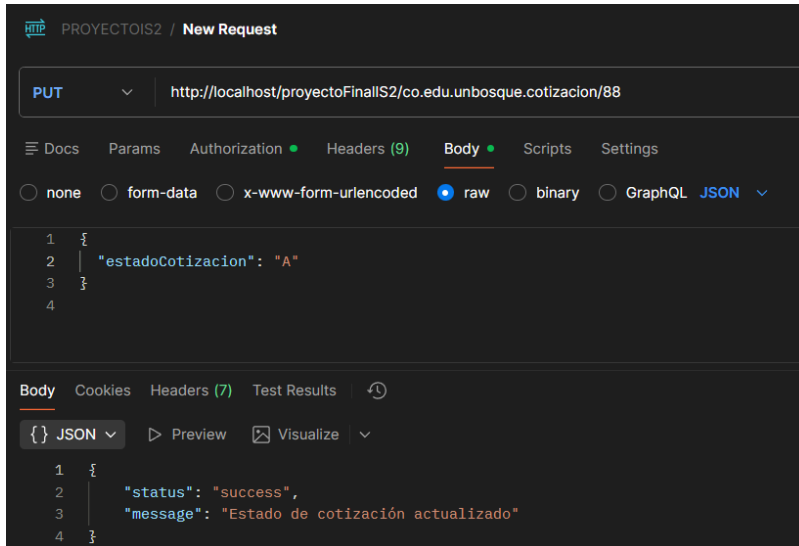


8. Cambiar el estado de una cotización a "A" (Aprobada)

Método HTTP: PUT

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.cotizacion/72>

Cuerpo de la solicitud (JSON) y resultado:

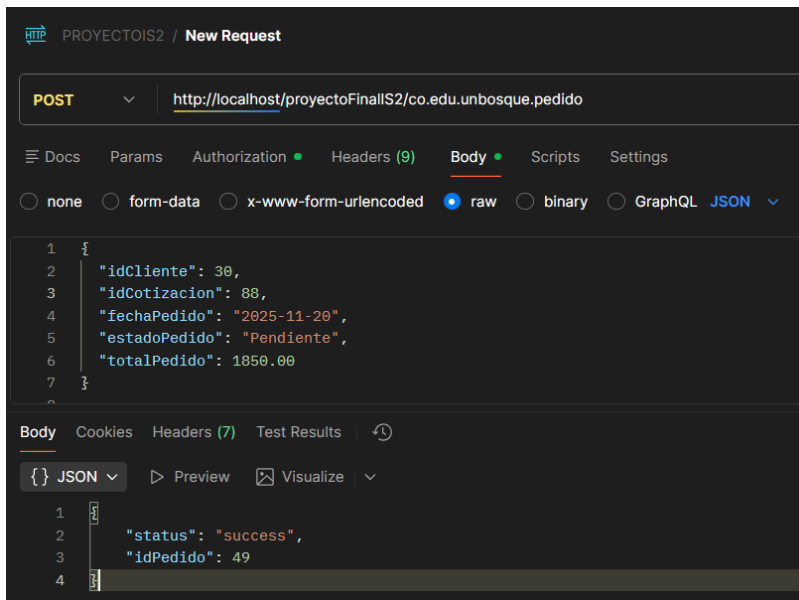


9. A partir de una cotización aprobada, crear un pedido

Método HTTP: POST

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.pedido>

Cuerpo de la solicitud (JSON) y resultado:

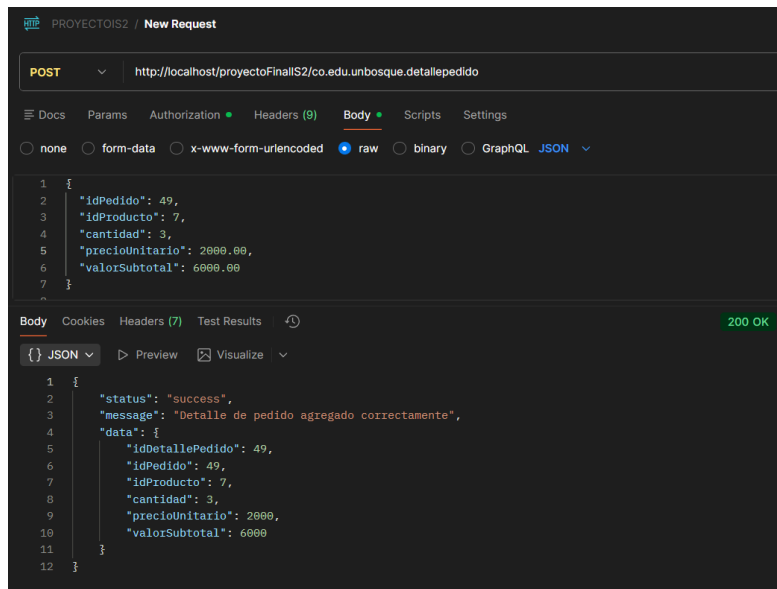


10. Añadir el detalle a un pedido

Método HTTP: POST

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.detallepedido>

Cuerpo de la solicitud (JSON):



11. Editar el detalle de un pedido

Método HTTP: PUT

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.pedido/46>

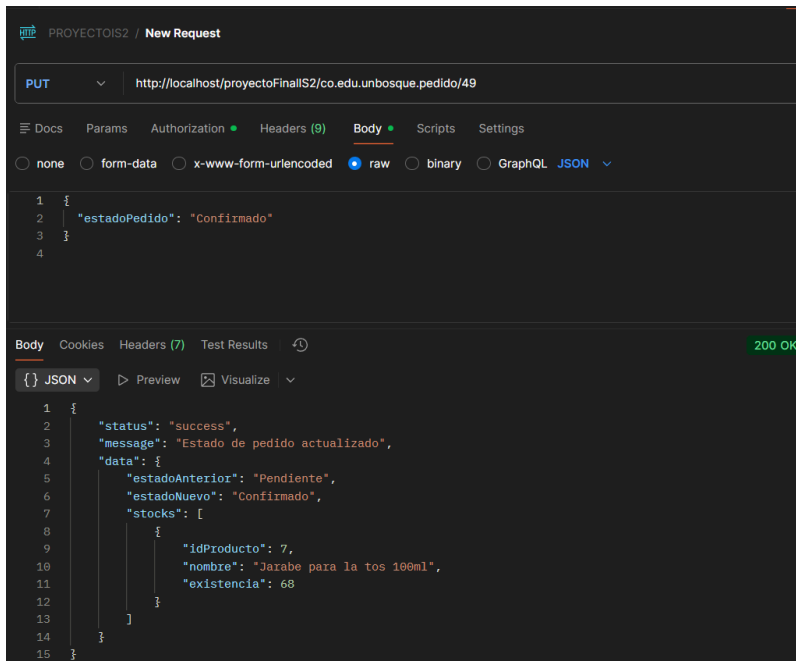
Cuerpo de la solicitud (JSON):

12. Cambiar el estado de un pedido a "Confirmado"

Método HTTP: PUT

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.pedido/49>

Cuerpo de la solicitud (JSON):

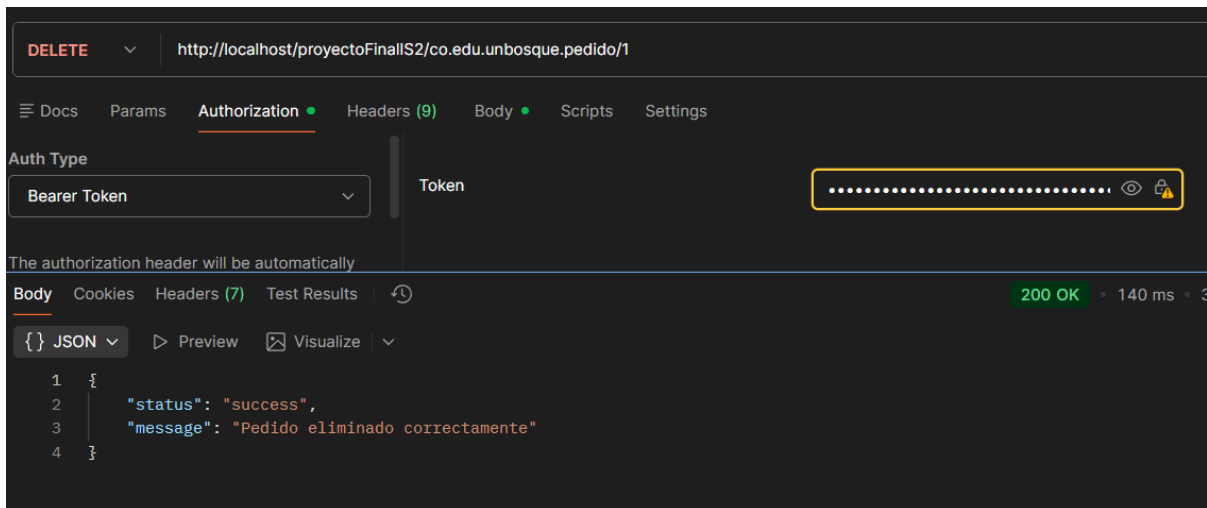


12. Eliminar un pedido

Método HTTP: DELETE

Ruta: <http://localhost/proyectoFinalIS2/co.edu.unbosque.pedido/49>

Cuerpo de la solicitud: No es necesario, solo la solicitud DELETE.



Cabe recalcar que todas las acciones en POSTMAN se realizaron con el token de Authorization generado automaticamente al iniciar sesión. En cada sesión se genera un nuevo token, para ese caso fue el siguiente:

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpYXQiOiJlM2MDQ1OTEsImV4cCI6MTc2MzYxNTM1MSwlc3ViIjoyLCJ1c2VybmFtZSI6InZlbnRlZG9yIiwicm9sZV9pZCI6Mn0.sBQfKjpVZFMBvi5n58U51ONnfWS33IY5Dy6bR7EmSGo

4. API Gateway

En este proyecto, no se está utilizando un API Gateway, por lo que no se requiere la configuración de enrutamiento centralizado ni la documentación de los endpoints expuestos por un gateway. En cambio, cada microservicio se comunica directamente con el cliente a través de sus propias rutas, gestionadas por el enrutador configurado en el archivo `index.php` mediante `FastRoute`.

a. Configuración de enrutamiento hacia los microservicios

Dado que no se está utilizando un API Gateway en esta arquitectura, cada microservicio tiene su propio enrutamiento. Las rutas definidas en el archivo `index.php` dirigen las solicitudes directamente a los controladores correspondientes, como `ClientController`, `CotizacionController`, `PedidoController`, entre otros. Por su parte, se hizo uso del enrutador **FastRoute** es el encargado de mapear las rutas de las solicitudes entrantes y ejecutar las funciones adecuadas para cada acción.

b. Seguridad básica (token, autenticación)

Aunque no se utiliza un API Gateway, se ha implementado seguridad básica a través de la autenticación JWT (JSON Web Token), que se utiliza para asegurar que las solicitudes a los microservicios sean realizadas por usuarios autenticados. El proceso de autenticación y autorización se lleva a cabo en los controladores de los servicios que requieren protección.

Flujo de autenticación en el proyecto:

Autenticación con JWT:

Cuando un usuario se loguea correctamente mediante el endpoint `/co.edu.unbosque.auth/login`, el sistema genera un token JWT que contiene la información del usuario autenticado, como su `idUsuario` y `rol`.

Este token JWT se almacena en el `localStorage` del cliente (navegador) y debe ser enviado en las cabeceras `Authorization` de todas las solicitudes que requieran autenticación.

Validación del token:

En cada microservicio que requiere autenticación, se verifica el token JWT enviado por el cliente. Este proceso se realiza utilizando el middleware `AuthMiddleware::validateJwtOrDie()`, que valida la firma y la expiración del token.

Si el token es válido, se permite el acceso a la solicitud y se extrae el `idUsuario` mediante `AuthMiddleware::getUserIdFromToken()`.

Si el token no es válido o no se proporciona, la solicitud es rechazada con un código de estado 401 (Unauthorized), lo que asegura que solo los usuarios autenticados puedan acceder a los recursos protegidos.

Autorización basada en roles:

Además de la autenticación, el sistema valida el rol del usuario (Únicamente el usuario Admin, y Vendedor) y le permite o restringe el acceso a las rutas según su rol. Esto se

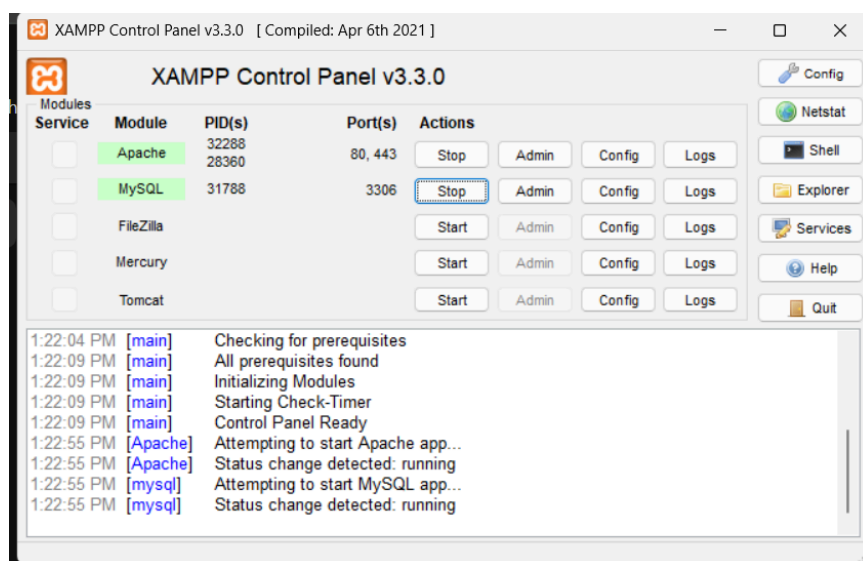
gestiona dentro de cada controlador, donde se revisa el rol del usuario extraído del token JWT.

c. Documentación de endpoints expuestos por el gateway (NO APLICA)

5. Despliegue y Acceso (en este ítem, el grupo deberá demostrar el funcionamiento del proyecto).

a. Microservicios desplegados (localmente, Render, Heroku, Máquinas virtuales, etc).

El proyecto se despliega localmente con ayuda de Xampp.



Por su parte, los microservicios están desplegados en una máquina virtual (MV) que corre en Debian 11 en donde vive el motor de base de datos PostgreSQL.

1. Iniciando la máquina virtual

- Para comenzar, la máquina virtual Debian 11 debe estar en ejecución. La máquina virtual está configurada con un adaptador de red tipo "Bridged Adapter" que permite que la MV se conecte a la red local y tenga una dirección IP accesible desde la red.
- La máquina virtual puede iniciarse desde un software de virtualización (como VirtualBox, VMware, etc.).
- Luego, se debe acceder al sistema con las credenciales de administrador para realizar las configuraciones necesarias.

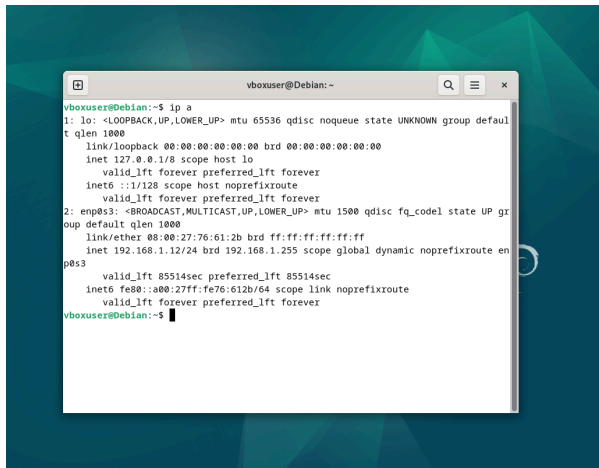
Usuario: postgres

Contraseña: 1234

2. Verificación de la IP de la máquina virtual (MV)

Para asegurarse de que la máquina virtual tenga una dirección IP accesible, se deben seguir los siguientes pasos:

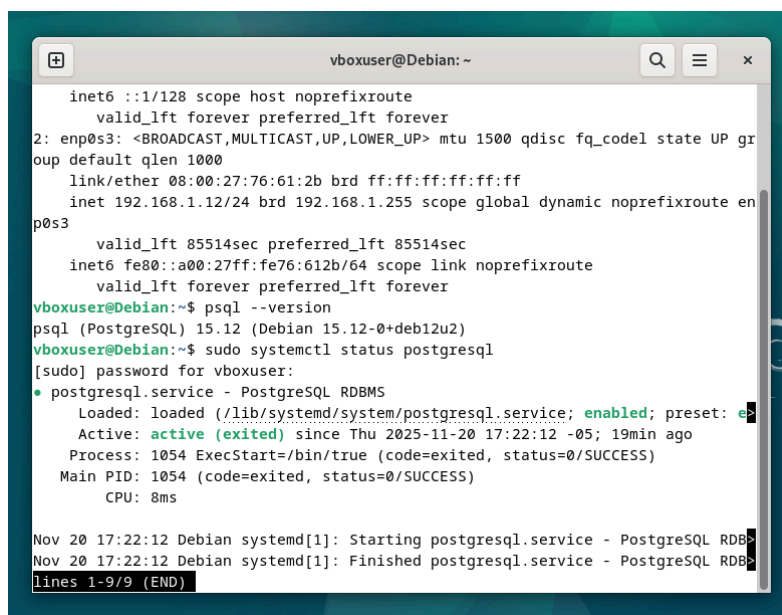
- Obtener la IP de la MV ejecutando el siguiente comando en el terminal de la máquina virtual: **ip a**



```
vboxuser@Debian:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:76:61:2b brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.12/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
        valid_lft 85514sec preferred_lft 85514sec
    inet6 fe80::a00:27ff:fe76:612b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
vboxuser@Debian:~$
```

Con la ip que se muestra en enp0s3 se puede acceder a los servicios de la MV desde el host en el que se esté trabajando el proyecto. El uso de la dirección ip se detalla más adelante en el punto b.

Posteriormente con el comando: `sudo systemctl start postgresql` y `sudo systemctl status postgresql` se debe comprobar si el motor de base de datos está corriendo en la MV.



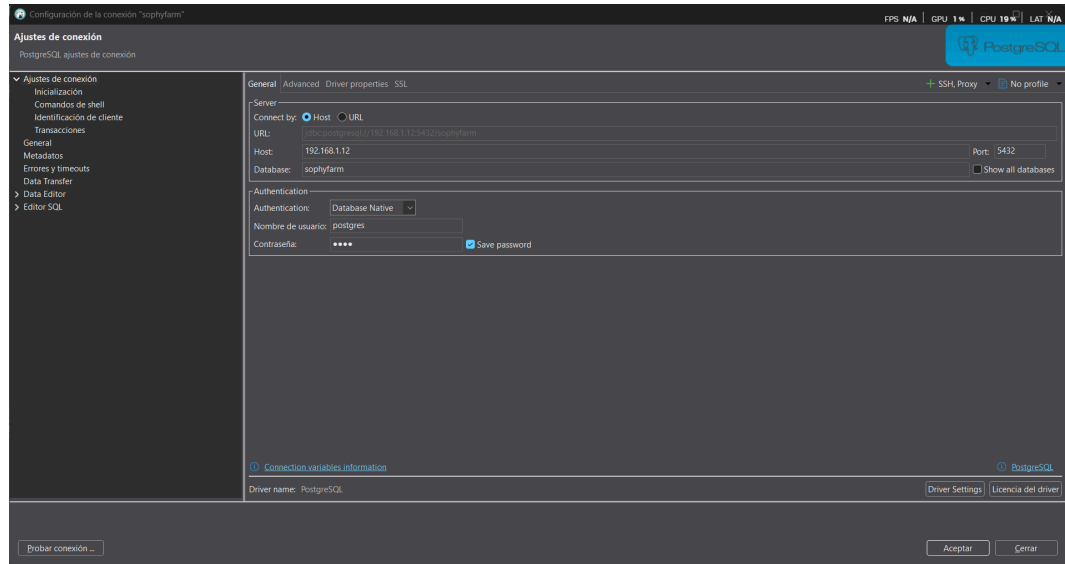
```
vboxuser@Debian:~$ ip a
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:76:61:2b brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.12/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
        valid_lft 85514sec preferred_lft 85514sec
    inet6 fe80::a00:27ff:fe76:612b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
vboxuser@Debian:~$ psql --version
psql (PostgreSQL) 15.12 (Debian 15.12-0+deb12u2)
vboxuser@Debian:~$ sudo systemctl status postgresql
[sudo] password for vboxuser:
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Thu 2025-11-20 17:22:12 -05; 19min ago
     Process: 1054 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
    Main PID: 1054 (code=exited, status=0/SUCCESS)
       CPU: 8ms

Nov 20 17:22:12 Debian systemd[1]: Starting postgresql.service - PostgreSQL RDBMS.
Nov 20 17:22:12 Debian systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
lines 1-9/9 (END)
```

b. Evidencia del funcionamiento (evidencias gráficas que deberá colocar en el informe a entregar).

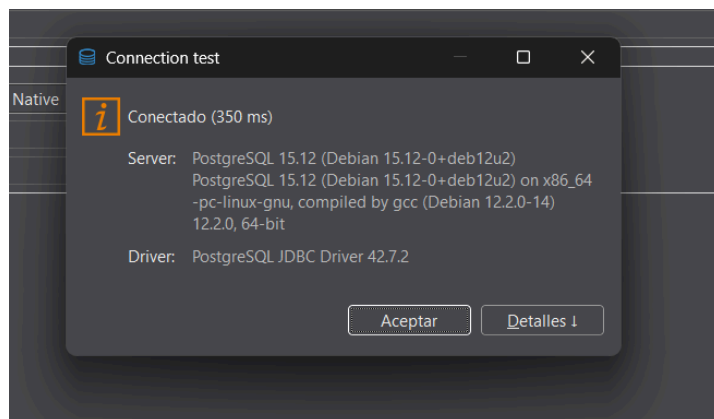
Inicialmente la MV debe estar corriendo luego de seguir los pasos de la anterior sección. Y luego de obtener la dirección ip ahora se debe configurar el entorno.

DBeaver: Se usó como herramienta de administración de la base de datos.



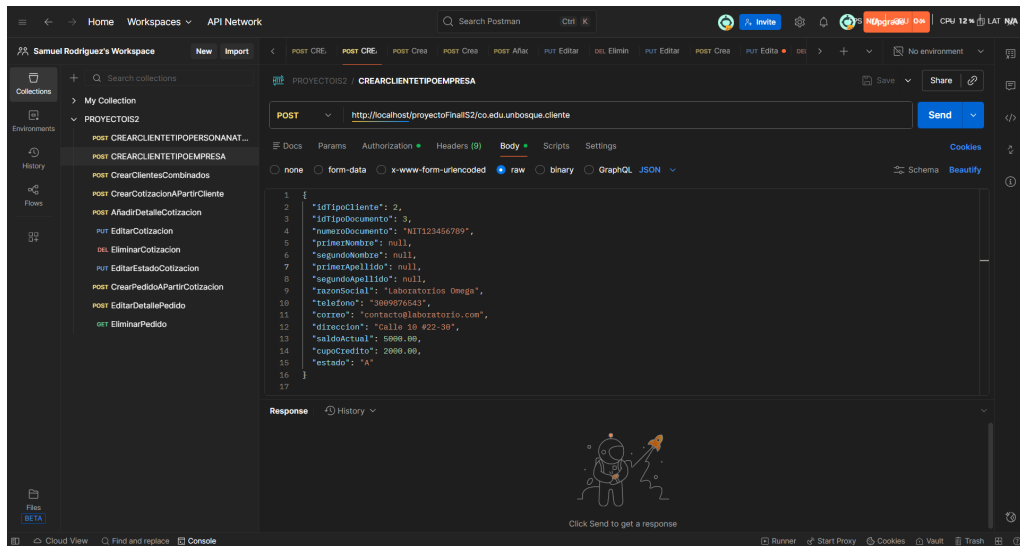
En esta sección se deben configurar los datos esenciales del proyecto con ayuda de:

- IP obtenida.
- Usuario y contraseña de la MV.
- Puerto de conexión.

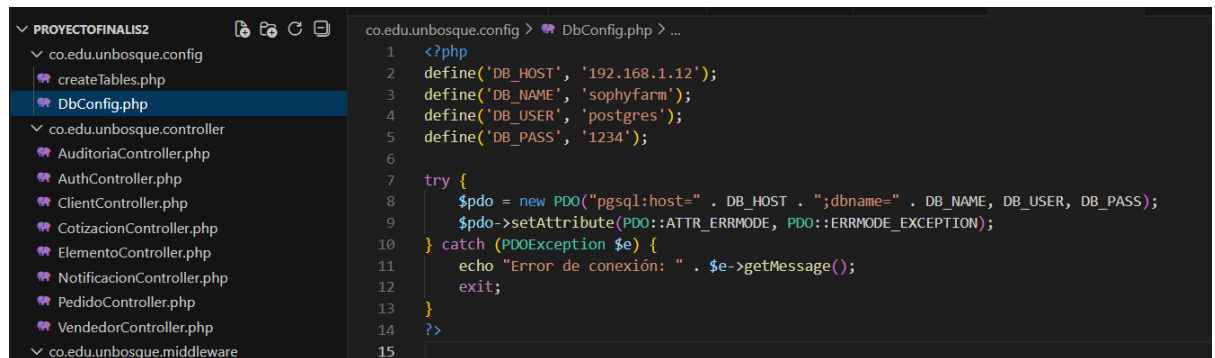


Si todo se configuró correctamente, al dar clic al botón probar conexión se debe obtener un mensaje de aprobación.

Postman: Se usó exclusivamente para realizar las pruebas del proyecto.



Luego se debe configurar el archivo DbConfig.php



En este se configura como tal la conexión a la base de datos desde el proyecto como tal.

Teniendo esto, ahora sí se puede iniciar con la ejecución del proyecto:

1. Iniciar sesión: Para este proceso cabe recalcar que hay dos usuarios acreedores al sistema:

-Usuario: Vendedor

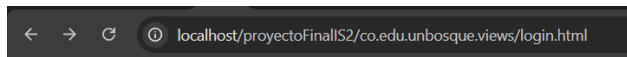
Contraseña: Vendedor123

-Usuario: Admin

Contraseña: Admin123

Inicialmente se iniciará sesión como vendedor por medio de la siguiente ruta:

<http://localhost/proyectoFinalIS2/co.edu.unbosque.views/login.html>

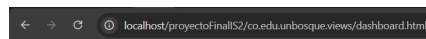


Login

Username:

Password:

Luego de iniciar sesión, este es el dashboard que debería ver el vendedor:



Dashboard

Bienvenido, vendedor

Tu rol es: Vendedor

Gestión de clientes

[Crear cliente](#)

[Listar / buscar clientes](#)

Gestión comercial

[Crear / gestionar cotizaciones](#)

[Crear / gestionar pedidos](#)

Reportes e indicadores (PDF)

[Ver / descargar reportes en PDF](#)

2. Crear un cliente.

Crear Cliente

Tipo de cliente:

Tipo de documento:

Número de documento:

Datos empresa

Razón social:

Teléfono:

Correo:

Dirección:

Saldo actual:

Cupo crédito:

Crear Cliente

Tipo de cliente:

Tipo de documento:

Número de documento:

Datos persona natural

Primer nombre:

Segundo nombre:

Primer apellido:

Segundo apellido:

Teléfono:

Correo:

Dirección:

Saldo actual:

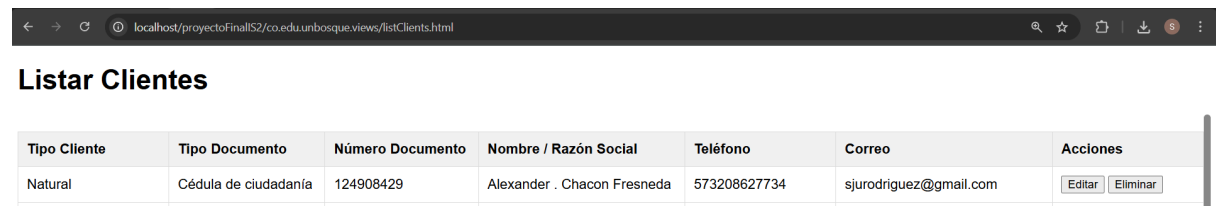
Cupo crédito:

Acá existe la posibilidad de crear ya sea un cliente tipo natural, o tipo empresa.

Se deben ingresar los datos correspondientes y si todo se ingreso correctamente se debería obtener un mensaje similar a este:

Cliente creado con ID: 44

3. Listar clientes



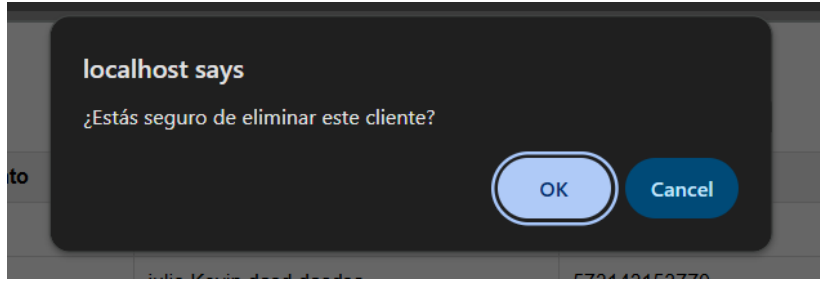
Tipo Cliente	Tipo Documento	Número Documento	Nombre / Razón Social	Teléfono	Correo	Acciones
Natural	Cédula de ciudadanía	124908429	Alexander . Chacon Fresneda	573208627734	sjurodriguez@gmail.com	Editar Eliminar

Posteriormente se vuelve al dashboard y se pulsa la opción listar/buscar clientes, en donde se encuentra una tabla con todos los clientes registrados en la empresa hasta el momento.

Acá existe la posibilidad de editar o eliminar un cliente.

Editar Cliente

Tipo Cliente	Persona Natural
Tipo Documento	Cédula de ciudadanía
Número Documento	124908429
Primer Nombre	Alexander
Segundo Nombre	.
Primer Apellido	Chacon
Segundo Apellido	Fresneda
Teléfono	573208627734
Correo	sjurodriguez@gmail.com
Dirección	Avenida Siempre Viva 10
Saldo Actual	210000.00
Cupo Crédito	210000.00
Actualizar Cliente	



4. Crear cotización a partir de un cliente registrado.

Posteriormente se vuelve al dashboard y se pulsa la opción de Crear/Gestionar cotizaciones.

Crear Cotización

Seleccionar Cliente: Alexander Chacon

Seleccionar Vendedor: Ana Martinez

Fecha de Cotización: 11/19/2025

Guardar Cotización

Cotizaciones Existentes

ID Cotización	Cliente	Vendedor	Fecha Cotización	Valor Neto	Estado	Acciones
90	42	7	2025-11-03	47124.00	Aprobada	Editar Eliminar Confirmar Enviar Email WhatsApp
89	43	6	2025-11-20	10710.00	Aprobada	Editar Eliminar Confirmar Enviar Email WhatsApp
88	41	9	2025-11-17	47124.00	Aprobada	Editar Eliminar Confirmar Enviar Email WhatsApp
86	30	6	2025-11-19	51836.40	Pendiente	Editar Eliminar Confirmar Enviar Email WhatsApp
72	30	8	2025-11-18	10710.00	Aprobada	Editar Eliminar Confirmar Enviar Email WhatsApp

Aquí se logran evidenciar las cotizaciones existentes y un formulario para crear una cotización.

Crear Cotización

Seleccionar Cliente: --Seleccione--

Seleccionar Vendedor: --Seleccione--

Fecha de Cotización: mm/dd/yyyy

Guardar Cotización

Cotización creada con ID: 91

Añadir Detalle

Detalles de la Cotización

Seleccionar Producto: Aspirina 500mg - \$1000.00

Cantidad: 22

Precio Unitario: 1000.00

Valor Subtotal: 22000.00

Guardar Detalle

Si la cotización se creó correctamente, se despliega otro formulario para añadir el detalle del pedido especificando los productos a cotizar y los precios.

Detalle de cotización añadido correctamente

Luego, en la tabla cotizaciones existentes existen las opciones de:

Editar, eliminar, confirmar (cambiar el estado de P (pendiente) por A (Aprobada)), enviar la cotización por Email y por WhatsApp.

Cotizaciones Existentes

ID Cotización	Cliente	Vendedor	Fecha Cotización	Valor Neto	Estado	Acciones
91	44	8	2025-11-18	23562.00	Pendiente	<button>Editar</button> <button>Eliminar</button> <button>Confirmar</button> <button>Enviar Email</button> <button>WhatsApp</button>
90	42	7	2025-11-03	47124.00	Aprobada	<button>Editar</button> <button>Eliminar</button> <button>Confirmar</button> <button>Enviar Email</button> <button>WhatsApp</button>
89	43	6	2025-11-20	10710.00	Aprobada	<button>Editar</button> <button>Eliminar</button> <button>Confirmar</button> <button>Enviar Email</button> <button>WhatsApp</button>
88	41	9	2025-11-17	47124.00	Aprobada	<button>Editar</button> <button>Eliminar</button> <button>Confirmar</button> <button>Enviar Email</button> <button>WhatsApp</button>
86	30	6	2025-11-19	51836.40	Pendiente	<button>Editar</button> <button>Eliminar</button> <button>Confirmar</button> <button>Enviar Email</button> <button>WhatsApp</button>
72	30	8	2025-11-18	10710.00	Aprobada	<button>Editar</button> <button>Eliminar</button> <button>Confirmar</button> <button>Enviar Email</button> <button>WhatsApp</button>

Este es el campo para editar y eliminar la cotización. Por su parte, existe la posibilidad de añadir más productos al detalle de la cotización, según desee el usuario.

Editar Cotización

ID Cotización

91

Valor Bruto

22000.00

ID Cliente

44

Valor Descuento

2200.00

ID Vendedor

8

Valor IVA

3762.00

Fecha Cotización

11/18/2025

Valor Neto

23562.00

Estado

Pendiente

Detalles de la Cotización

ID Detalle	ID Producto	Cantidad	Precio Unitario	Subtotal	Acciones
76	6	22	1000.00	22000.00	<button>Actualizar</button> <button>Eliminar</button>

Agregar nuevo detalle

Producto

--Seleccione--

Cantidad

Precio Unitario

Subtotal

Agregar Detalle

localhost says

¿Estás seguro de eliminar esta cotización?

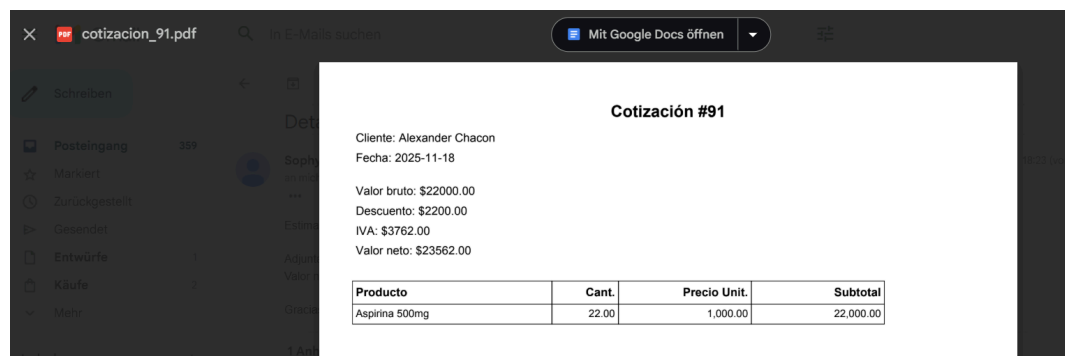
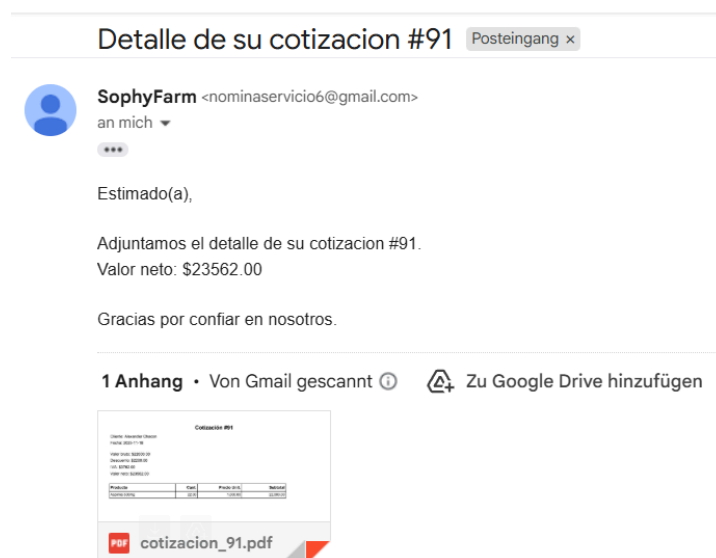
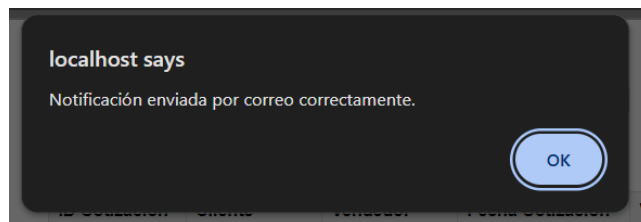
OK

Cancel

Cabe recalcar que solo las cotizaciones en estado “Aprobado” pueden convertirse en un pedido. Luego de pulsar el botón confirmar debe cambiar el estado.

ID Cotización	Cliente	Vendedor	Fecha Cotización	Valor Neto	Estado	Acciones
91	44	8	2025-11-18	23562.00	Aprobada	<button>Editar</button> <button>Eliminar</button> <button>Confirmar</button> <button>Enviar Email</button> <button>WhatsApp</button>

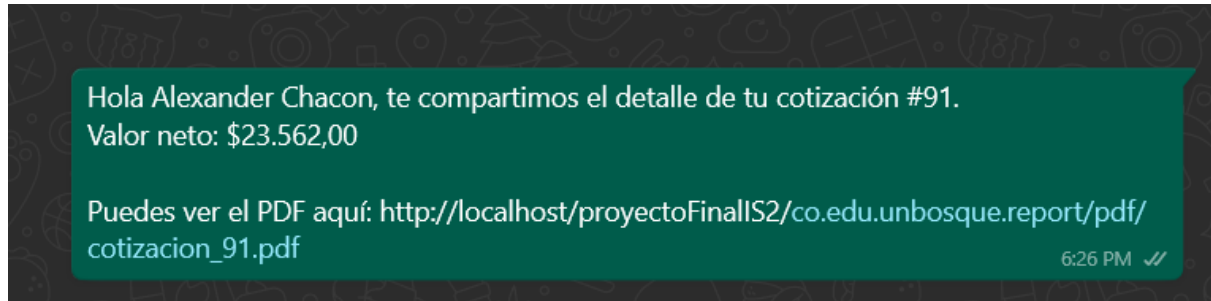
Si la cotización se aprobó y el usuario desea ver su cotización por correo electrónico se hace click a “Enviar Email”



Si el proceso fue exitoso, debe enviarse el correo al destinatario con el asunto de la cotización realizada, y un archivo pdf con la información detallada.

Luego, se pulsa la opción para hacer el envío a WhatsApp.

Si el proceso fue exitoso debe enviarse la información al destinatario y cabe recalcar que para el archivo pdf, se optó por una solución que permite que solo el vendedor desde el sistema pueda mostrarle el archivo pdf al cliente por medio de una ruta.



5. Crear un pedido a partir de una cotización existente.

Para continuar con el proceso, se debe volver al Dashboard y pulsar el botón Crear / gestionar pedidos.

Acá se tienen exactamente las mismas opciones que con las cotizaciones.

Gestión de Pedidos

Crear Pedido

Seleccionar Cotización: --Seleccione--

Fecha de Pedido: mm/dd/yyyy

Guardar Pedido

Pedidos Existentes

ID Pedido	Cliente	ID Cotización	Fecha Pedido	Total Pedido	Estado	Acciones
66	asd dasdas	90	2025-11-04	0.00	Pendiente	Editar Eliminar Confirmar Cancelar Enviar Email
65	asd dasdas	90	2025-11-04	163500.00	Cancelado	Editar Eliminar Confirmar Cancelar Enviar Email
64	asd dasdas	90	2025-11-03	218000.00	Pendiente	Editar Eliminar Confirmar Cancelar Enviar Email
63	asd dasdas	90	2025-11-12	191840.00	Cancelado	Editar Eliminar Confirmar Cancelar Enviar Email
62	asd dasdas	90	2025-11-11	490500.00	Cancelado	Editar Eliminar Confirmar Cancelar Enviar Email
61	asd dasdas	90	2025-11-04	0.00	Pendiente	Editar Eliminar Confirmar Cancelar Enviar Email

Luego de crear un pedido, seleccionando la cotización previa y la fecha se debe añadir el detalle del pedido.

Gestión de Pedidos

Crear Pedido

Seleccionar Cotización: --Seleccione--

Fecha de Pedido: mm/dd/yyyy

Guardar Pedido

Stock insuficiente para Aspirina 500mg. Stock actual: 170, solicitado: 500

Añadir Detalle

Agregar Detalle al Pedido

Seleccionar Producto: Aspirina 500mg

Cantidad: 500

Precio Unitario: 1000.00

Valor Subtotal: 500000.00

Guardar Detalle

El proceso aquí sí varía con respecto a las cotizaciones, debido a que si se cotiza un producto del cual no hay todo el stock disponible, debido a que aún es una cotización existe la posibilidad de que ese stock faltante se logre reponer antes de realizar el pedido.

A diferencia en el proceso de los pedidos, si es obligatorio que el stock que solicite el cliente se encuentre disponible, o si no se obtendrá la advertencia en color rojo que se logra evidenciar en la imagen detectando stock insuficiente.

Pedidos Existentes

ID Pedido	Cliente	ID Cotización	Fecha Pedido	Total Pedido	Estado	Acciones
67	Alexander Chacon	91	2025-11-20	158050.00	Pendiente	Editar Eliminar Confirmar Cancelar Enviar Email WhatsApp
						Editar Eliminar Confirmar Cancelar Enviar Email

Editar Pedido

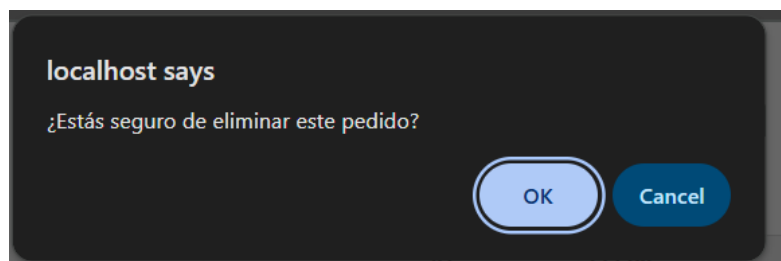
ID Pedido: 67
Cliente: Alexander Chacon
ID Cotización: 91
Fecha Pedido: 2025-11-20
Total Pedido: 158050.00
Estado: Pendiente

Detalles del Pedido

ID Detalle	Producto	Cantidad	Precio Unitario	Subtotal	Acciones
64	Aspirina 500mg	145	1000.00	145000.00	Actualizar Eliminar

Agregar nuevo detalle

Producto: [Seleccione--] Cantidad: [] Precio Unitario: [] Subtotal: [] Agregar Detalle

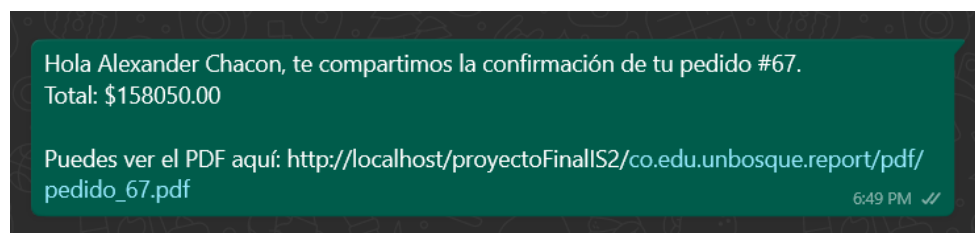
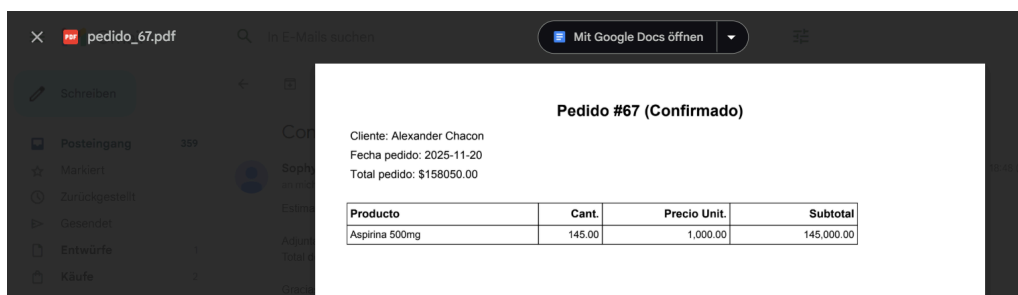
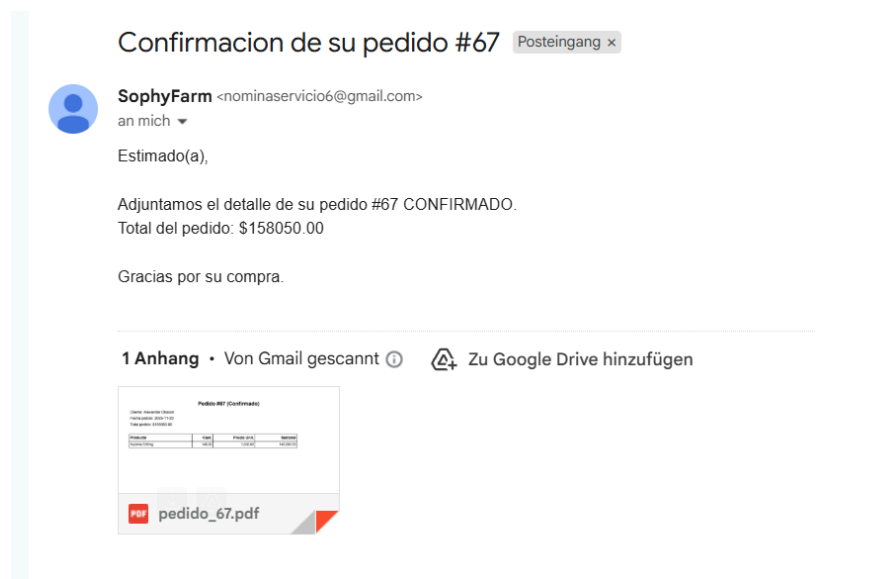
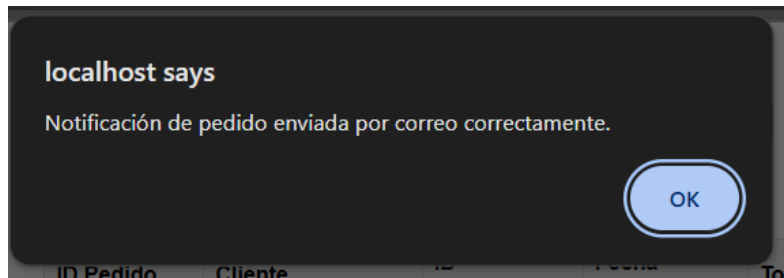


Luego de añadir el detalle del pedido se puede editar, eliminar, confirmar (Cambiar su estado de “Pendiente” a “Confirmado”, cancelar pedido, enviar el pedido al Email registrado y enviar el WhatsApp al número registrado.

Al crear un pedido siempre está en estado Pendiente, luego de cambiar su estado a Confirmado, ahora si se descuenta el stock.

ID Pedido	Cliente	ID Cotización	Fecha Pedido	Total Pedido	Estado	Acciones
67	Alexander Chacon	91	2025-11-20	158050.00	Confirmado	Editar Eliminar Confirmar Cancelar Enviar Email WhatsApp

Posteriormente, el envío del pedido por correo electrónico y a WhatsApp funciona con la misma lógica que en las cotizaciones.



6. Generar los reportes

Finalmente, se vuelve al dashboard y por parte del usuario vendedor, la última opción disponible que posee es: **Ver / descargar reportes en PDF.**

Reportes del sistema

Usuario: vendedor
Rol: Vendedor
Vendedor: acceso a reportes comerciales

[Volver al Dashboard](#) [Cerrar sesión](#)

¿Qué vas a encontrar aquí?

En esta sección puedes generar reportes en PDF que ayudan a la administración a entender mejor el comportamiento de clientes, cotizaciones, pedidos e inventario.

- Cualquier usuario con rol **Administrador** o **Vendedor** puede descargar los reportes.
- Los reportes se generan en tiempo real a partir de la información registrada en el sistema.
- Al abrir un reporte se descargará o mostrará directamente el PDF en una nueva pestaña.

Nota: la auditoría (historial de acciones) solo está disponible para el administrador desde el Dashboard.

1. Clientes naturales vs empresas

Enfoque: Clientes

Muestra cuántos clientes son persona natural y cuántos son empresa (razón social).

- Ayuda a saber si el negocio está más orientado a retail o a clientes corporativos.
- Útil para definir campañas, ofertas y tipo de comunicación.

[Ver reporte en PDF](#) Fuente: tabla cliente

2. Cotizaciones vs pedidos

Enfoque: Conversión

Compara la cantidad total de cotizaciones con la cantidad total de pedidos.

- Permite estimar qué tanto se convierten las cotizaciones en pedidos.
- Si hay muchas cotizaciones y pocos pedidos, algo está fallando en precio, seguimiento o servicio.

[Ver reporte en PDF](#) Fuente: cotización / pedido

3. TOP productos cotizados vs pedidos confirmados

Enfoque: Productos

Compara los productos más cotizados con los productos más presentes en pedidos confirmados.

- Detecta productos muy cotizados pero poco pedidos (posibles problemas de precio o stock).
- Identifica productos estratégicos que son tanto cotizados como confirmados.

[Ver reporte en PDF](#) Fuente: detalle_cotizacion / detalle_pedido / elemento

4. Pedidos cancelados

Enfoque: Riesgo

Lista cuántos pedidos han sido cancelados por cliente y el valor asociado.

- Sirve para detectar clientes que piden con frecuencia y luego cancelan.
- Ayuda a entender el impacto económico de las cancelaciones.

[Ver reporte en PDF](#) Fuente: pedido / cliente (+ vendedor opcional)

5. Resumen de pedidos confirmados

Enfoque: Volumen

Resume los pedidos en estado Confirmado: cantidad, total, promedio, pedido más alto y más bajo.

- Muestra el volumen que efectivamente llega a la etapa de confirmación.
- Ayuda a estimar el "ticket promedio" de los pedidos confirmados.

[Ver reporte en PDF](#) Fuente: pedido (estado Confirmado)

6. Pedidos confirmados por mes

Enfoque: Tendencia

Agrupar los pedidos confirmados por mes, mostrando cantidad y valor total.

- Ayuda a ver si el número de pedidos confirmados va en aumento, estable o en caída.
- Útil para identificar meses fuertes y meses débiles.

[Ver reporte en PDF](#) Fuente: pedido (Confirmado, agrupado por mes)

7. Rotación de inventario por producto

Enfoque: Inventario

Muestra por cada producto: cantidad total vendida (en pedidos confirmados) y stock actual.

- Clasifica los productos en alta, media, baja rotación o sin rotación.
- Ayuda a tomar decisiones sobre compras, promociones y posibles discontinuaciones.

[Ver reporte en PDF](#) Fuente: elemento / detalle_pedido / pedido

Para beneficio del negocio, se decidió plantear 7 reportes de valor para tomar decisiones dentro de la empresa, los cuales son los siguientes:

1. Reporte de clientes naturales vs empresas (razón social).
2. Reporte de cantidad de cotizaciones confirmadas vs cantidad pedidos confirmados.
3. Reporte del top de los productos cotizados vs productos pedidos (confirmados).
4. Reporte de los pedidos cancelados.
5. Reporte del resumen de los pedidos confirmados.
6. Reporte de los pedidos confirmados por mes.
7. Reporte de la rotación del inventario por productos (es decir cuáles fueron los pedidos más pedidos y confirmados)

7. Iniciar sesión como administrador

Para continuar con el proceso se pulsa el botón cerrar sesión y posteriormente se hace el login con las credenciales únicas del administrador.

Login

Username:

Password:

Login

Por su parte, el administrador solo posee acceso a consultar los reportes de valor para el negocio, y analizarlos.

Aparte, este usuario es el único que tiene la capacidad de consultar el historial o auditoría del sistema, en donde se evidencian las acciones realizadas de los dos usuarios.

Dashboard

Bienvenido, admin

Tu rol es: Administrador

Cerrar sesión

Reportes e indicadores (PDF)

[Ver / descargar reportes en PDF](#)

Historial / Auditoría del sistema

[Ver historial de operaciones](#)

La sección de reportes es exactamente igual que como con el usuario “Vendedor”.

Reportes del sistema

Usuario: admin
Rol: Administrador
Administrador: acceso total a reportes

Volver al Dashboard

Cerrar sesión

¿Qué vas a encontrar aquí?

En esta sección puedes generar reportes en PDF que ayudan a la administración a entender mejor el comportamiento de clientes, cotizaciones, pedidos e inventario.

- Cualquier usuario con rol **Administrador** o **Vendedor** puede descargar los reportes.
- Los reportes se generan en tiempo real a partir de la información registrada en el sistema.
- Al abrir un reporte se descargará o mostrará directamente el PDF en una nueva pestaña.

Nota: la auditoría (historial de acciones) solo está disponible para el administrador desde el Dashboard.

1. Clientes naturales vs empresas

Enfoque: Clientes

Muestra cuántos clientes son persona natural y cuántos son empresa (razón social).

- Ayuda a saber si el negocio está más orientado a retail o a clientes corporativos.
- Útil para definir campañas, ofertas y tipo de comunicación.

Ver reporte en PDF

Fuente: tabla cliente

2. Cotizaciones vs pedidos

Enfoque: Conversión

Compara la cantidad total de cotizaciones con la cantidad total de pedidos.

- Permite estimar qué tanto se convierten las cotizaciones en pedidos.
- Si hay muchas cotizaciones y pocos pedidos, algo está fallando en precio, seguimiento o servicio.

Ver reporte en PDF

Fuente: cotización / pedido

3. TOP productos cotizados vs pedidos confirmados

Enfoque: Productos

Compara los productos más cotizados con los productos más presentes en pedidos confirmados.

- Detecta productos muy cotizados pero poco pedidos (posibles problemas de precio o stock).
- Identifica productos estratégicos que son tanto cotizados como confirmados.

Ver reporte en PDF

Fuente: detalle_cotizacion / detalle_pedido / elemento

4. Pedidos cancelados

Enfoque: Riesgo

Lista cuántos pedidos han sido cancelados por cliente y el valor asociado.

- Sirve para detectar clientes que piden con frecuencia y luego cancelan.
- Ayuda a entender el impacto económico de las cancelaciones.

Ver reporte en PDF

Fuente: pedido / cliente (+ vendedor opcional)

5. Resumen de pedidos confirmados

Enfoque: Volumen

Resume los pedidos en estado Confirmado: cantidad, total, promedio, pedido más alto y más bajo.

- Muestra el volumen que efectivamente llega a la etapa de confirmación.
- Ayuda a estimar el "ticket promedio" de los pedidos confirmados.

Ver reporte en PDF

Fuente: pedido (estado Confirmado)

6. Pedidos confirmados por mes

Enfoque: Tendencia

Agrupar los pedidos confirmados por mes, mostrando cantidad y valor total.

- Ayuda a ver si el número de pedidos confirmados va en aumento, estable o en caída.
- Útil para identificar meses fuertes y meses débiles.

Ver reporte en PDF

Fuente: pedido (Confirmado, agrupado por mes)

7. Rotación de inventario por producto

Enfoque: Inventario

Muestra por cada producto: cantidad total vendida (en pedidos confirmados) y stock actual.

- Clasifica los productos en alta, media, baja rotación o sin rotación.
- Ayuda a tomar decisiones sobre compras, promociones y posibles discontinuaciones.

Ver reporte en PDF

Fuente: elemento / detalle_pedido / pedido

Por su parte, el botón de “Ver historial de operaciones” muestra el historial de todas las acciones que han realizado los usuarios separando por, inserciones, actualizaciones, eliminaciones, e inicio de sesiones, aplicando un filtro por la fecha que se desee.

Historial / Auditoría del sistema

Solo los usuarios con rol Administrador pueden ver esta sección.

Usuario: Todos

Acción: Todas

Desde: mm/dd/yyyy

Hasta: mm/dd/yyyy

Filtrar

Limpiar filtros

Fecha / hora	Usuario	Acción	Descripción
2025-11-20 19:20:24.719257	admin		Inicio de sesión exitoso para el usuario admin
2025-11-20 18:46:48.513257	vendedor	Actualizar	Cambio de estado del pedido #67: Pendiente -> Confirmado. Stocks afectados: Aspirina 500mg (ID 6) stock actual: 25
2025-11-20 18:44:52.36596	vendedor	Eliminar	Eliminó la cotización #93
2025-11-20 18:44:25.947223	vendedor	Actualizar	Actualizó estado de la cotización #93 a A
2025-11-20 18:44:14.95212	vendedor	Actualizar	Recalculó totales de la cotización #93 (Bruto: 66000.00, Descuento: 6600.00, IVA: 11286.00, Neto: 70686.00)
2025-11-20 18:44:06.493666	vendedor	Inserir	Creó cotización #93 para el cliente 40 (vendedor 9, fecha 2025-11-10)
2025-11-20 18:42:17.052663	vendedor	Eliminar	Eliminó la cotización #92
2025-11-20 18:42:14.568958	vendedor	Actualizar	Recalculó totales de la cotización #92 (Bruto: 3000.00, Descuento: 300.00, IVA: 513.00, Neto: 3213.00)
2025-11-20 18:42:11.965695	vendedor	Inserir	Creó cotización #92 para el cliente 40 (vendedor 9, fecha 2025-11-24)
2025-11-20 18:41:55.723141	vendedor	Eliminar	Eliminó la cotización #86
2025-11-20 18:41:46.968497	vendedor	Eliminar	Eliminación del pedido #52 (cliente #42, cotización #90)
2025-11-20 18:32:16.691341	vendedor	Inserir	Creación de pedido #67 asociado a la cotización #91 para el cliente #44
2025-11-20 18:21:11.746014	vendedor	Actualizar	Actualizó estado de la cotización #91 a A

Historial / Auditoría del sistema

Solo los usuarios con rol **Administrador** pueden ver esta sección.

Usuario:

Todos

 Acción:

Insertar

 Desde:

11/20/2025

 Hasta:

11/20/2025

Filtrar

Limpiar filtros

Fecha / hora	Usuario	Acción	Descripción
2025-11-20 18:44:06.493666	vendedor	Insertar	Creó cotización #93 para el cliente 40 (vendedor 9, fecha 2025-11-10)
2025-11-20 18:42:11.965695	vendedor	Insertar	Creó cotización #92 para el cliente 40 (vendedor 9, fecha 2025-11-24)
2025-11-20 18:32:16.691341	vendedor	Insertar	Creación de pedido #67 asociado a la cotización #91 para el cliente #44
2025-11-20 18:15:49.888638	vendedor	Insertar	Creó cotización #91 para el cliente 44 (vendedor 8, fecha 2025-11-18)
2025-11-20 18:07:59.675485	vendedor	Insertar	Creó cliente ID 44 (Persona natural, documento: 124908429)
2025-11-20 15:23:58.186579	vendedor	Insertar	Creación de pedido #66 asociado a la cotización #90 para el cliente #42
2025-11-20 15:23:22.997148	vendedor	Insertar	Creación de pedido #65 asociado a la cotización #90 para el cliente #42
2025-11-20 15:18:28.559869	vendedor	Insertar	Creación de pedido #64 asociado a la cotización #90 para el cliente #42
2025-11-20 15:15:49.176628	vendedor	Insertar	Creación de pedido #63 asociado a la cotización #90 para el cliente #42
2025-11-20 15:14:27.41788	vendedor	Insertar	Creación de pedido #62 asociado a la cotización #90 para el cliente #42
2025-11-20 15:13:55.412771	vendedor	Insertar	Creación de pedido #61 asociado a la cotización #90 para el cliente #42
2025-11-20 15:10:50.14537	vendedor	Insertar	Creación de pedido #60 asociado a la cotización #90 para el cliente #42
2025-11-20 15:08:26.366912	vendedor	Insertar	Creación de pedido #59 asociado a la cotización #90 para el cliente #42

Historial / Auditoría del sistema

Solo los usuarios con rol **Administrador** pueden ver esta sección.

Usuario:

Todos

 Acción:

Actualizar

 Desde:

11/20/2025

 Hasta:

11/20/2025

Filtrar

Limpiar filtros

Fecha / hora	Usuario	Acción	Descripción
2025-11-20 18:46:48.513257	vendedor	Actualizar	Cambio de estado del pedido #67: Pendiente -> Confirmado. Stocks afectados: Aspirina 500mg (ID 6) stock actual: 25
2025-11-20 18:44:25.947223	vendedor	Actualizar	Actualizó estado de la cotización #93 a A
2025-11-20 18:44:14.95212	vendedor	Actualizar	Recalculó totales de la cotización #93 (Bruto: 66000.00, Descuento: 6600.00, IVA: 11286.00, Neto: 70686.00)
2025-11-20 18:42:14.568558	vendedor	Actualizar	Recalculó totales de la cotización #92 (Bruto: 3000.00, Descuento: 300.00, IVA: 513.00, Neto: 3213.00)
2025-11-20 18:21:11.746014	vendedor	Actualizar	Actualizó estado de la cotización #91 a A
2025-11-20 18:17:15.997967	vendedor	Actualizar	Recalculó totales de la cotización #91 (Bruto: 22000.00, Descuento: 2200.00, IVA: 3762.00, Neto: 23562.00)
2025-11-20 15:23:37.5818	vendedor	Actualizar	Cambio de estado del pedido #65: Confirmado -> Cancelado. Stocks afectados: Paracetamol 500mg (ID 1) stock actual: 75
2025-11-20 15:23:34.234768	vendedor	Actualizar	Cambio de estado del pedido #65: Pendiente -> Confirmado. Stocks afectados: Paracetamol 500mg (ID 1) stock actual: 0
2025-11-20 15:16:28.131645	vendedor	Actualizar	Cambio de estado del pedido #63: Confirmado -> Cancelado. Stocks afectados: Vitamina C 1000mg (ID 5) stock actual: 134
2025-11-20 15:16:24.056341	vendedor	Actualizar	Cambio de estado del pedido #63: Pendiente -> Confirmado. Stocks afectados: Vitamina C 1000mg (ID 5) stock actual: 112
2025-11-20 15:14:50.973525	vendedor	Actualizar	Cambio de estado del pedido #62: Pendiente -> Cancelado
2025-11-20 15:11:03.409379	vendedor	Actualizar	Cambio de estado del pedido #60: -> Confirmado. Stocks afectados: Producto #0 (ID 0) stock actual: 0
2025-11-20 15:08:47.71627	vendedor	Actualizar	Cambio de estado del pedido #59: Pendiente -> Cancelado

Historial / Auditoría del sistema

Solo los usuarios con rol **Administrador** pueden ver esta sección.

Usuario:

Todos

 Acción:

Eliminar

 Desde:

11/12/2025

 Hasta:

11/20/2025

Filtrar

Limpiar filtros

Fecha / hora	Usuario	Acción	Descripción
2025-11-20 18:44:52.36596	vendedor	Eliminar	Eliminó la cotización #93
2025-11-20 18:42:17.052663	vendedor	Eliminar	Eliminó la cotización #92
2025-11-20 18:41:55.723141	vendedor	Eliminar	Eliminó la cotización #66
2025-11-20 18:41:46.968497	vendedor	Eliminar	Eliminación del pedido #52 (cliente #42, cotización #90)
2025-11-19 23:16:25.579894	vendedor	Eliminar	Eliminación del pedido #1 (cliente #ND, cotización #ND)
2025-11-19 22:55:14.147818	vendedor	Eliminar	Eliminó la cotización #87
2025-11-19 18:21:09.323189	vendedor	Eliminar	Eliminó la cotización #76
2025-11-19 18:21:06.010629	vendedor	Eliminar	Eliminó la cotización #78
2025-11-19 18:20:59.829198	vendedor	Eliminar	Eliminó cliente ID 36
2025-11-19 18:13:52.780599	vendedor	Eliminar	Eliminó la cotización #79
2025-11-19 18:13:50.69494	vendedor	Eliminar	Eliminó la cotización #80
2025-11-19 18:13:40.252207	vendedor	Eliminar	Eliminación del pedido #48 (cliente #33, cotización #76)
2025-11-19 18:12:46.791923	vendedor	Eliminar	Eliminó la cotización #81

6. Documentación:

a. Manual de Usuario (si existe interfaz gráfica).

Recaltar que el manual de usuario se entregó por medio del archivo .txt llamado “ManualUsuarioSophyFarm(CotizacionesYPedidos).txt”

b. Manual de instalación y despliegue.

Por su parte, el manual de instalación y despliegue se entregó por medio del archivo llamado “ManualInstalaciónYDespliegueMV(CotizacionesYPedidos).txt”.

7. Estimación del producto

Para el proceso de estimación se hizo una consulta a 3 de los docentes indicados.

Firmas:

Andres Rey (15 años de experiencia)

Cotiza- ciones	CRUD COTIZ.	1 SEMANA	Media baja.	Tecnología falta comunicación No tener repositorio	MZ +10.
Pedidos.	Generar un pedido a partir de una cotización.	2 semanas	Medio alto.	Tecnología falta comunicación No tener repositorio	

Samuel Galindo Pinzón (10 años de experiencia)

Microservicio de Cotizaciones (CRUD completo)

Supuesto: CRUD estándar (crear, consultar, actualizar, eliminar), validaciones básicas, sin reglas de negocio complejas.

Fase / Actividad	Descripción	Horas estimadas
Análisis y entendimiento	Definir campos, flujos, dependencias, reglas	4 h
Diseño de API y arquitectura	Endpoints, contratos, manejo de errores	3 h
Infraestructura (configuración mínima)	Variables, permisos, despliegue inicial	4 h
Desarrollo	CRUD completo + validaciones	10 h
Pruebas (unitarias e integración)	Verificar funcionamiento interno	5 h
Estabilización	Ajustes, correcciones post-pruebas	4 h
Total estimado		30 horas

Microservicio de Pedidos (CRUD completo con dependencia obligatoria de una cotización)

Supuesto: CRUD estándar que requiere validar que la cotización exista. Implica integración obligatoria.

Fase / Actividad	Descripción resumida	Horas estimadas
Análisis y entendimiento	Reglas del pedido, validación contra cotizaciones	5 h
Diseño de API y arquitectura	Endpoints, contratos, relación con cotizaciones	4 h
Infraestructura	Configuración + colas/eventos si aplica	4 h
Desarrollo	CRUD + validación obligatoria + manejo de errores	12 h
Pruebas (unitarias, integración y dependencia)	Especial foco en la relación con cotizaciones	6 h
Estabilización	Ajustes post-pruebas e integración	5 h
Total estimado		36 horas

Microservicio Complejidad Justificación

Cotizaciones	Baja – Media	Es un CRUD directo, sin dependencias externas obligatorias y con reglas de negocio poco complejas.
Pedidos	Media	Aunque es un CRUD, tiene dependencia obligatoria de otro microservicio , lo que añade lógica de validación, manejo de errores entre servicios y casos de integración.

Principales riesgos

Microservicio de Cotizaciones

- Diseños incompletos del modelo de datos, generando retrabajo.
- Manejo insuficiente de validaciones, lo que provoca datos inconsistentes.
- Errores en el versionado de la API (si se requiere evolucionar).

Microservicio de Pedidos

- Dependencia obligatoria del microservicio de Cotizaciones:
 - Fallos en la comunicación entre servicios.
 - Problemas si Cotizaciones está caído o responde lento.
 - Errores por IDs inexistentes o mal enviados.
- Inconsistencias transaccionales, si no se manejan adecuadamente los estados.
- Mayor complejidad en pruebas, especialmente en escenarios de falla o latencia.

Riesgos comunes a ambos

- Configuraciones de infraestructura incompletas o mal gestionadas.
- Falta de pruebas de integración.
- Dificultad para monitorear o rastrear errores si no hay logs centralizados.

Nombre del experto: SAMUEL GALINDO PINZON

Firma: _____

Christian Duarte (7 años de experiencia)

CONSTANCIA DE APROBACIÓN

Yo, Christian Felipe Dark, identificado con CC N.º 1013605851,
dejo constancia de que el día 18/11/2025 estuve reunido con el estudiante **Samuel Julián Rodríguez Chávez**, con quien se realizó una estimación del módulo cotizaciones y pedidos para el proyecto final de la asignatura Ingeniería de Software II por medio de la técnica "juicio de expertos".

Teniendo en cuenta lo anterior, y habiendo revisado la información presentada durante la reunión, apruebo la firma y validación para continuar con el proceso del proyecto.

Firma del docente:



Nombre del docente:

Christian Felipe Dark

Cargo / Área:

Dark

Fecha:

18/11/2025

Tabla:

Microservicio	Funcionalidad principal	Estimación (horas)	Nivel de complejidad	Riesgos	Experto consultado
Cotizaciones	CRUD cotizaciones	40 horas	Media baja	Tecnología Mala comunicación No tener repositorio	Andrés Rey
Pedidos	Crear un pedido a partir de una cotización	80 horas	Medio alto	Tecnología Mala comunicación No tener repositorio	Andrés Rey
Cotizaciones	CRUD cotizaciones	30 horas	Media baja	Diseños incompletos del modelo de datos, generando retrabajo. • Manejo insuficiente de validaciones, lo que provoca datos inconsistentes. • Errores en el versionado de la API (si se requiere evolucionar).	Samuel Galindo

Pedidos	Crear un pedido a partir de una cotización	36 horas	Media	Dependencia obligatoria del microservicio de Cotizaciones: ➤ Fallos en la comunicación entre servicios. ➤ Problemas si Cotizaciones está caído o responde lento. ➤ Errores por IDs inexistentes o mal enviados. • Inconsistencias transaccionales, si no se manejan adecuadamente los estados. • Mayor complejidad en pruebas, especialmente en escenarios de falla o latencia.	Samuel Galindo
Cotizaciones	CRUD cotizaciones	Fase de análisis y diseño: Horas Optimistas: 3 h Probables: 5 h Pesimistas: 8 h Fase desarrollo: Horas Optimistas: 4 h Probables: 6 h Pesimistas: 10 h Pruebas: Horas: Optimistas: 3 horas Probable: 5 horas Pesimista: 8 horas Total: 14.15 h	Bajo	Ausencia de personal o reemplazar un recurso, el recurso no es el mismo que inició con el que terminamos Una vez analizados los requerimientos, puede que cambien las reglas del proceso en tiempos de ejecución, y se cambie lo que querían (reajustar). Revisar la integración con otros módulos (que sea viable ya sea por microservicios o etc, que se pueda realizar).	Christian Duarte
Pedidos	Crear un pedido a partir de una cotización	Fase de análisis y diseño: Horas Optimistas: 10 h Probables: 14 h Pesimistas: 15 h	Medio-alto	Ausencia de personal o reemplazar un recurso, el recurso no es el mismo que inició con el que terminamos	Christian Duarte

		Fase desarrollo: Horas Optimistas: 15 h Probables: 25 h Pesimistas: 30 h Pruebas: Horas: Optimistas:30 h Probable: 45 h Pesimista: 65 h Pruebas integración: Horas: Optimistas: 5 h Probable: 7 h Pesimista: 12 h Total: 73.46 h		Una vez analizados los requerimientos, puede que cambien las reglas del proceso en tiempos de ejecución, y se cambie lo que querían (reajustar). Revisar la integración con otros módulos (que sea viable ya sea por microservicios o etc, que se pueda realizar).	
--	--	---	--	---	--

Para la estimación con el experto Christian Duarte se debe aplicar la técnica de estimación planning poker para calcular la cantidad de horas totales.

Para este cálculo se hizo uso de la ecuación de la distribución beta:
 $(hpe + 4 * ho + hp)/6$

Funcionalidad principal: CRUD cotizaciones

Fase de análisis y diseño: $(8 + 4 * 3 + 5) / 6 = 4.16$ h

Fase desarrollo: $(10 + 4 * 4 + 6) / 6 = 5.33$ h

Pruebas: $(8 + 4 * 3 + 8) / 6 = 4.66$ h

Total = 4.16 h + 5.33 h + 4.66 h = 14.15 h

Funcionalidad principal: Crear un pedido a partir de una cotización

Fase de análisis y diseño: $(15 + 4 * 10 + 14) / 6 = 11.5$ h

Fase desarrollo: $(30 + 4 * 15 + 25) / 6 = 19.16$ h

Pruebas: $(65 + 4 * 30 + 45) / 6 = 36.3$ h

Integración: $(12 + 4 * 5 + 7) / 6 = 6.5$ h

Total = 4.16 h + 5.33 h + 4.66 h = 14.15 h

Juicio de expertos:

Experto	Estimación (horas)
1	66 h
2	120 h
3	88 h

- Experto 1 (experiencia media): peso 0.3
- Experto 2 (más experiencia): peso 0.5
- Experto 3 (menos experiencia): peso 0.2
- Estimación final: $66 \times 0.3 + 120 \times 0.5 + 88 \times 0.2 = 19.8 + 60 + 17.6 = 97.4$ horas